

SOURCE-DRIVEN ARCHITECTURE / C++20

TaskflowLite

实现原理与调度算法

从原子发布、工作窃取，到依赖推进与内存回收

PUBLISH

工作如何可见

CLAIM

谁取得执行权

COMPLETE

何时完成与回收

60 节源码解析 / 48 幅矢量图解 / 关键状态逐步演算

有界双端队列 · 分片共享栈 · Notifier · join · Semaphore · ObjectPool

v3.2.0 workspace snapshot / baseline 91424ac + local changes

2026-09-23 · 以当前启用源码为准 · 含源码定位与验证边界

阅读约定与证据边界

本书按“存储与发布 → 取得执行权 → 依赖推进 → 完成与回收”展开。第 05-15 节剖析两种工作容器，第 16-24 节解释调度与休眠，第 25-41 节解释图、异步依赖和嵌套执行，第 42-54 节解释资源、异常与内存，第 55-60 节给出完整轨迹、取舍和验证入口。

每节末尾的“源码”均为本地快照中的文件与行号；路径默认相对 `taskflowlite/core/`。行号帮助定位，函数名才是后续版本中稳定的检索入口。末尾另附源码指纹。所有“算法摘要”均为解释用伪代码，保留关键顺序，不是可直接替换原实现的补丁。

蓝色框强调关键状态，青色箭头表示数据、引用或执行责任的方向（以各图标注为准），橙色表示竞争窗口或必须满足的条件。 时间图从上向下读；环形图中的数字为逻辑索引，物理槽位另行标出。图内的简写 `rlx/acq/rel/ar/sc` 分别表示 `relaxed/acquire/release/acq_rel/seq_cst`。

“源码事实”表示代码直接做了什么；“设计解释”表示根据协议推导出的目的；“前提”表示调用约束或尚需形式化证明的条件。本文不把注释中的“无锁”“安全”“高性能”当成已验证结论，也不把一次压力测试视为 C++ 内存模型证明。

当前工作区的共享容器是 `SharedWorkStack`，原 `UnboundedQueue` 已删除。`pipeline.hpp` 只有注释内容；`notifier.hpp` 后半部的分组位图版本也在注释中，均不属于本书描述的运行路径。

本书的完整性指覆盖当前调度核心和支撑组件的主要实现协议。API 的参数表、安装方法和入门示例见同目录《完整使用手册》；D2 导出只负责展示图结构，不参与运行时调度。

目录 / 01

01 从四个问题理解整个执行器	6
02 Work 的物理结构与三种关系	7
03 原子操作分别解决什么问题	8
04 为什么需要两种工作容器	9
05 有界队列：逻辑区间与物理环	10
06 push：先填槽，再发布边界	11
07 pop：多数情况只动尾部	12
08 最后一项：读到同一指针也只能一人赢	13
09 steal：一次尝试，而非阻塞出队	14
10 为什么槽位也必须是 atomic	15
11 批量 push 与溢出移交	16
12 共享工作栈：三个字段，两个世界	17
13 共享栈生产者：计数为什么先增加	18
14 共享栈消费者：gate 的取得与释放	19
15 共享栈到底按什么顺序取工作	20
16 从提交到执行：三条快慢路径	21
17 窃取策略：记忆、随机化与退让	22
18 睡眠之前为什么必须再检查队列	23
19 Notifier：把等待拆成票据与停泊	24
20 prepare：怎样得到自己的等待票号	25

目录 / 02

21 commit 与 cancel: 同一张票的两种结局	26
22 notify: 先消费预等待, 再摘等待栈	27
23 丢唤醒: 两个线程不能同时 “没看见”	28
24 park/unpark: 封住最后一个睡眠窗口	29
25 图边如何保存, 为什么分前后两段	30
26 初始化图: 物理入度不等于 join weight	31
27 join: 最后一个到达者取得后继	32
28 父 slot: 完成计数究竟在数什么	33
29 cache 接力: 为什么线性链不必反复入队	34
30 Branch 与 Jump 是两种执行协议	35
31 循环与重复运行: 两个不同层次	36
32 异步状态: 一个控制字里的四类信息	37
33 异步依赖的 +1 哨兵: 封住提前启动	38
34 动态登记与前驱完成如何互斥	39
35 Finished 发布之后, 收尾仍未全部结束	40
36 引用释放为什么采用迭代回收	41
37 Invoker: 统一框架, 不同执行阶段	42
38 Runtime: 用哨兵分离函数返回与任务完成	43
39 SubFlow 与 Module: 动态图和复用图	44
40 协作等待: 等孩子时继续帮执行器工作	45

目录 / 03

41 TaskGroup: 把一段子任务收拢到局部锚点	46
42 多信号量获取: 全检查后统一扣减	47
43 为什么必须最后解锁 blocker	48
44 release: 整链唤醒, 重新竞争资源	49
45 SpinMutex: 短锁也需要减少争用写	50
46 异常如何寻找归档位置	51
47 停止请求是协作标记, 不是线程终止	52
48 ResultSlot 与 Future: 存储不负责同步	53
49 Payload: 把调用快路径与冷操作分开	54
50 对象池: 块、slab 与分片	55
51 Tagged head: ABA 为什么需要版本	56
52 对象池快路径与慢路径的成本	57
53 SmallVector 与随机数: 辅助结构也有算法	58
54 缓存行、配置与平台前提	59
55 全过程演算: 菱形图的每一份责任	60
56 全过程演算: 异步依赖加资源等待	61
57 成本模型: 优化省掉了什么, 又增加什么	62
58 已知边界与架构审查重点	63
59 如何验证这些解释, 而不制造虚假信心	64
60 源码阅读路线与术语索引	65
附录: 源码指纹与再生成说明	

01 从四个问题理解整个执行器

一个任务系统必须回答四个独立问题：工作放在哪里，谁有权执行，何时可以推进后继，以及何时所有相关对象都能销毁。TaskflowLite 用不同机制分别回答它们，避免让一个全局互斥锁同时保护整个系统。

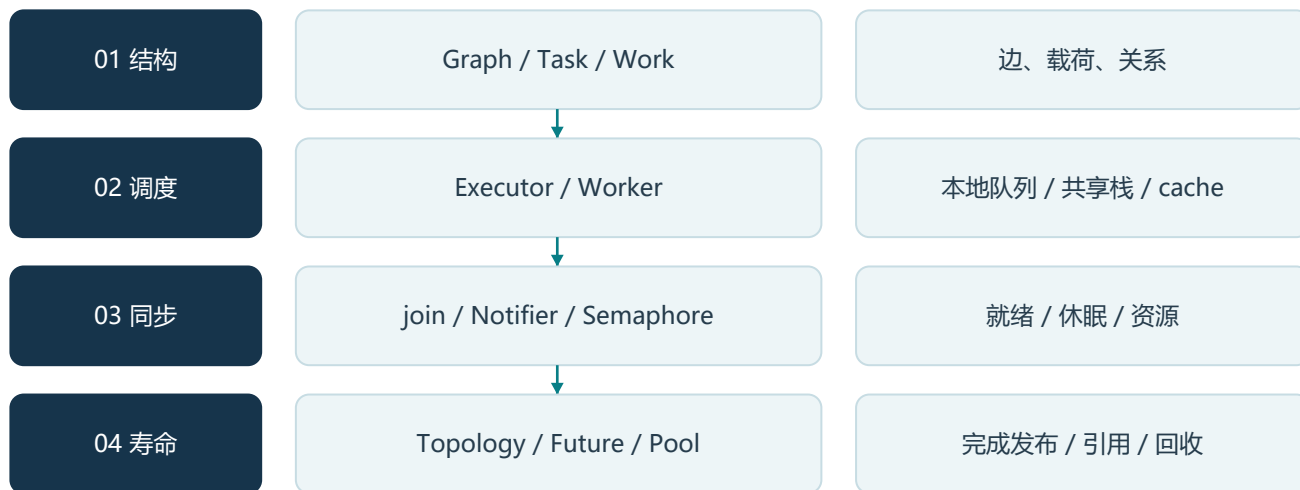


图 01：调度核心的四层职责

结构层。 Graph 管理图节点，Work 保存边、载荷、依赖计数与父指针。Task 是图节点的操作句柄；异步句柄则通过引用计数延长对应 Work 的寿命。图拓扑决定逻辑依赖，队列位置决定当前由谁尝试执行，两者不能等同。

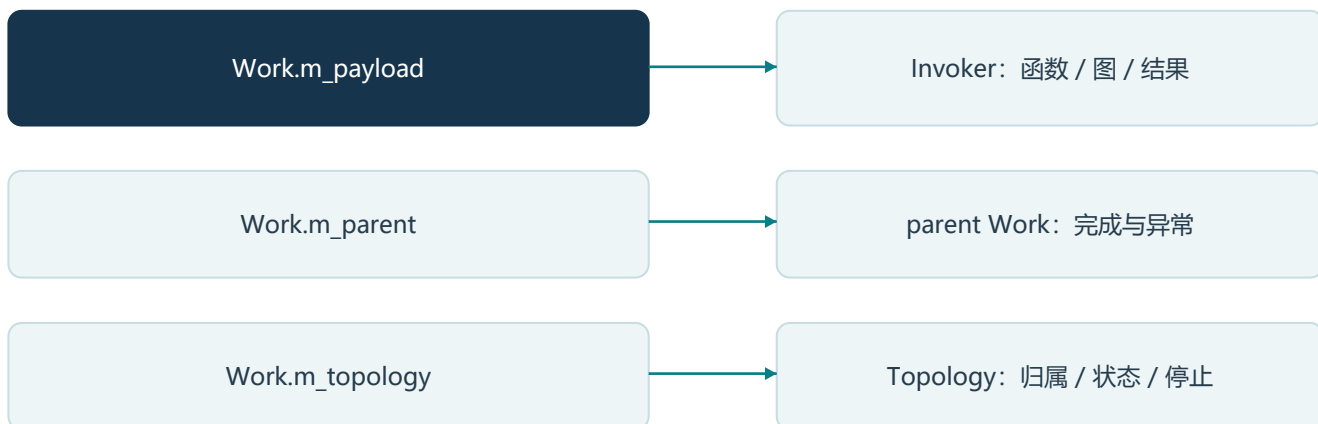
运行层。 Executor 建立 Worker 池。每个 Worker 有一个单所有者的有界双端队列；外部提交和本地溢出进入分片共享工作栈。第一个就绪后继可以进入 cache，由当前 Worker 直接接力执行。

同步层。 就绪传播依靠 join 计数；休眠依靠 Notifier；信号量约束资源。一次唤醒只表示应再次找工作，既不指定某个 Work，也不证明队列非空。

生命周期层。 Topology 保存执行器归属、状态、停止位和强引用计数。Finished 发布结果可见性；最后一份引用决定回收。任务“已完成”和内存“可销毁”是两个不同事件。

源码：executor.hpp:845,1113,1682,1847; work.hpp:550; topology.hpp:63。

02 Work 的物理结构与三种关系



另有 edges、join、flags 和 m_next；图示为引用关系，不是 ABI 偏移。

图 02：Work、载荷、运行拓扑及三种连接

Work 是调度器传递的实际对象。它包含类型擦除的 Payload、边数组、m_num_successors、原子 m_join_counter、控制位、图归属、父 Work、Topology 指针以及侵入式链接 m_next。观察者和信号量配置属于附加数据。图不是靠任务对象之间的虚函数继承来统一执行，而是通过 Payload 的调用入口分派。

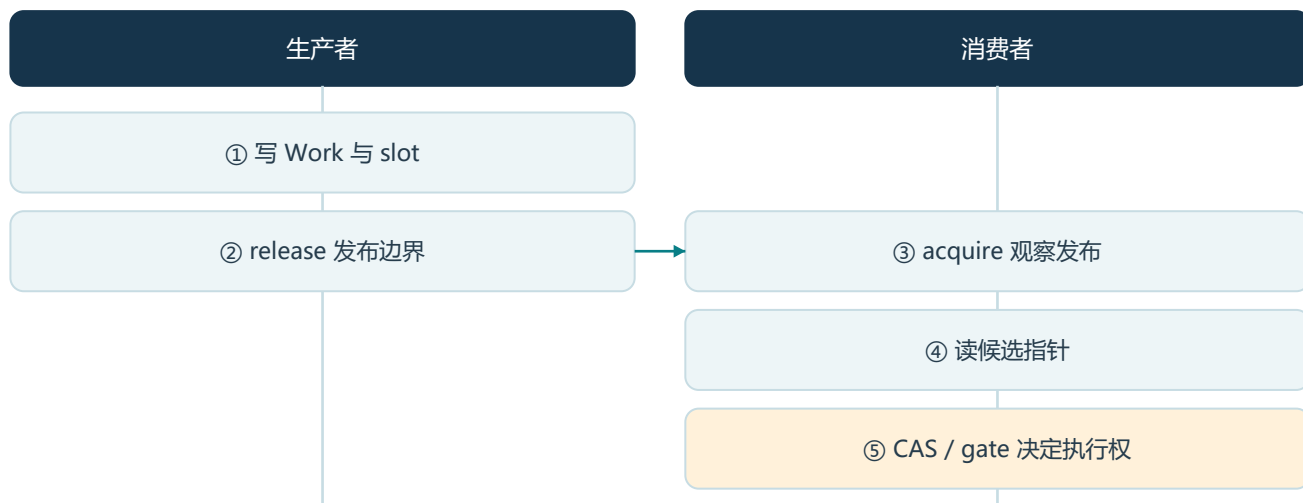
必须区分三种关系。**依赖边**说明前驱何时允许后继执行；**Work 父链**用于完成计数和异常寻找锚点；**Topology 父链**用于停止请求的继承。一个 Runtime 子任务即便不继承父 Topology，仍可计入父 Work 的完成计数。

m_next 是非拥有、可复用的链接槽：在共享工作栈里串工作，在信号量等待链里串等待者。一个 Work 不能同时位于两条需要此字段的链。取出共享工作、从信号量重新发布时，必须先保存 next，再清空链接，才可交给下一种容器。

Work 中有非原子字段并不意味着它们可任意并发访问。安全性来自阶段所有权：构造期独占、发布后执行权唯一、动态边受控制字中的 LOCKED 保护、回收期仅最后一个引用释放者操作。每个字段都应追问“当前阶段谁能写”，而不是简单地全部改成 atomic。

源码：work.hpp:276-562；shared_work_stack.hpp:228-234；executor.hpp:1757。

03 原子操作分别解决什么问题



看见指针、取得执行权、允许回收，分别由不同协议保证。

图 03：数据发布与执行权竞争是两条协议

普通写入并不会自动对另一线程建立先行发生关系。典型发布过程是：生产者先写对象和槽位，再做 release；消费者通过相匹配的 acquire 观察发布，之后才能依赖那些普通写入。这里的“观察”要求满足 C++ 的同步关系，不能解释成 acquire 会刷新所有内存。

relaxed 保证该原子对象的访问不发生数据竞争，并保留其修改顺序；它不负责独立发布其他字段。

release/acquire 负责跨线程传递已完成的写入。**acq_rel RMW** 同时取得先前到达者的结果并继续发布自己的结果，适合 join 计数链。**seq_cst fence/CAS** 在需要协调不同原子变量观察顺序的位置提供更强约束。

成功读取一个 Work 指针只解决“看见”的问题，未必取得执行权。有界队列窃取者先读槽位，再竞争 top 的 CAS；失败者必须丢弃指针。共享栈消费者先取得 gate，才可操作非原子的私有链头。

本文把“线性化点”用于具有明确抽象操作语义的竞争事件，例如成功窃取的 top CAS。empty()、共享栈预留计数和失败窃取属于保守快照或尝试接口，不能强行解释成一个全局 FIFO 的精确状态。

设计解释：对每个原子操作单独写“为了线程安全”没有帮助。应建立三张表：谁发布对象、谁裁决归属、谁发布完成；本书随后逐一对应实际指令顺序。

源码：bounded_queue.hpp:188-292；shared_work_stack.hpp:121-234；executor.hpp:1334,1635。

04 为什么需要两种工作容器

维度	Worker 本地有界队列	分片共享工作栈
写入方	只有所属 Worker	外部线程及本地溢出路径
读取方	Owner pop, 其他 Worker steal	多个 Worker 尝试 steal
顺序	Owner 从尾取, 窃取者从头取	incoming 栈与已分离链共同决定
容量	固定, 默认 1024 个指针	无固定槽位上限, 复用 Work 链接
争用位置	窃取者竞争 top	生产者竞争 incoming, 消费者竞争 gate

本地队列把最常见的“当前 Worker 产生工作再执行工作”压缩为短路径。Owner 取最新工作，常有利于保持近期数据和调用链的缓存局部性；窃取者拿较旧工作，可把另一段计算交给空闲核心。这里是调度倾向，不是性能或公平性的保证。

共享工作栈解决多生产者发布：外部线程不是任何本地队列的 Owner，不能直接调用其 push。本地固定容量不足时也需要一个后备入口，否则要么阻塞，要么丢工作。侵入式链使用 Work 已有的字段，入栈本身不另分配链表节点。

这两类容器都只传递指针，不自动持有引用。发布前必须先建立执行期寿命和父完成计数。否则“队列线程安全”仍挡不住悬空 Work 或提前完成的父任务。

共享栈的消费者区有互斥 gate，因此不应把整个系统称为“全路径无锁”。固定环也不等于系统有背压：溢出继续进入共享层，持续提交仍可增加 Work 与图对象的内存占用。

源码：worker.hpp; macros.hpp:219; executor.hpp:1712-1742; shared_work_stack.hpp:113。

05 有界队列：逻辑区间与物理环

逻辑区间: [6, 11) top=6 bottom=11



物理槽 = 逻辑索引 & 7; 不是让 int64 逻辑索引任意溢出。

图 05: 容量 8, 逻辑区间 [6,11) 映射到物理槽

队列保存单调演进的有符号 64 位逻辑索引 top 与 bottom。稳定状态下有效逻辑区间为 [top, bottom), 数量为差值; 物理位置用 $\text{index} \& (\text{cap}-1)$ 计算。因此容量必须是大于 1 的二次幂。图中的逻辑 8、9、10 分别落在物理槽 0、1、2。

Owner 是唯一修改 bottom 的线程; top 由窃取者及争夺最后一项的 Owner 通过 CAS 推进。多个消费者因此可以共享同一个逻辑头部, 而不需要让每次普通 Owner pop 都去修改同一热点原子。

槽位本身是 `atomic<Tp>`。它存的是指针, 不是任务对象; 同一物理槽可以在不同轮次保存不同 Work。缓存数组、top、bottom 分别按缓存行尺寸对齐, 但这并不意味着每个数组元素都独占缓存行。

`size()` 与 `empty()` 使用独立的 relaxed 快照; Owner pop 期间还会暂时把 bottom 往回移。它们适合调度探测, 不是与操作绑定的事务条件。“先判非空再取”仍可能取空, “检查未满后另一个线程 push”也不能扩大成多 Owner 协议。

前提: 逻辑索引必须在 `int64_t` 可表示范围内。掩码只让物理槽回绕, 不会使有符号逻辑索引溢出合法。容器也不负责元素销毁和调用方寿命。

源码: `bounded_queue.hpp:121-175`。

06 push: 先填槽，再发布边界

算法摘写 / 保留关键操作顺序

```
b = bottom.load(rlx)
t = top.load(acq)
if b - t >= capacity: overflow(value); return
slot[b & mask].store(value, rlx)
bottom.store(b + 1, rel)
```

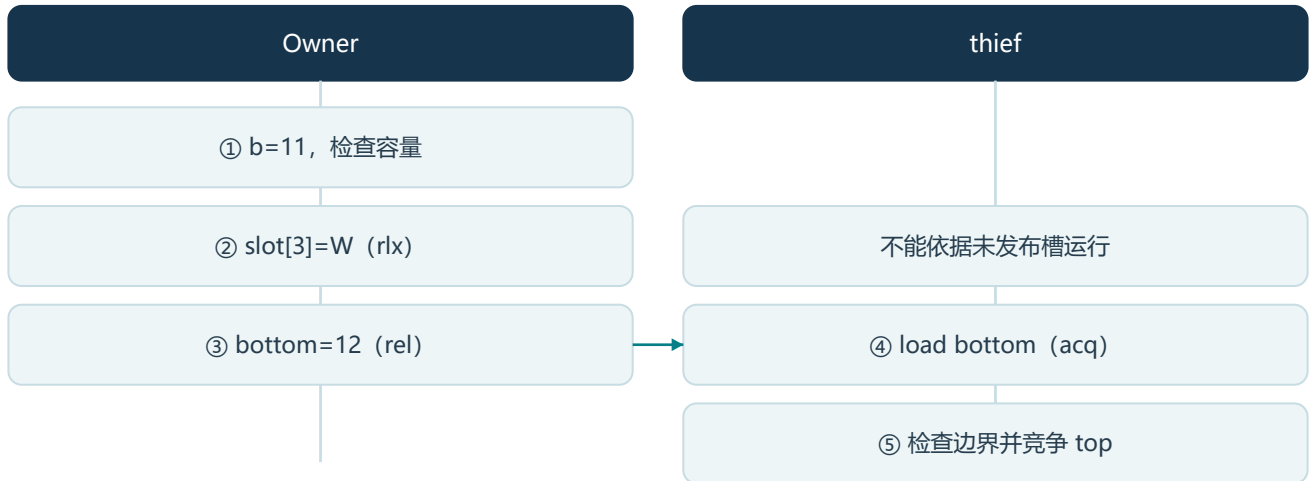


图 06: push 的发布边界

检查容量时，Owner 不会与另一个 Owner 争夺尾槽。窃取者只会推进 top、释放空间，因而观察到较旧的 top 最多导致偏保守的溢出判断。真正把新槽纳入逻辑有效区间的是最后的 bottom release store。

先写槽再增加 bottom 至关重要。若次序反过来，窃取者可能观察到非空，却读到该轮尚未写入的指针。代码的消费者路径通过 bottom acquire 配合此发布；slot 的 relaxed 访问仍是原子访问，用于处理槽位复用，不能简单替换成普通指针数组。

满队列时调用传入的 overflow 回调。本地调度器提供的回调把 Work 推到共享栈，因此当前 push 不等待别人取走元素；独立使用 BoundedQueue 的用户则自行决定溢出策略。这个回调属于容器外的行为，其成本与抛异常能力不能被算作“固定时间入队”。

这里的设计取舍是固定空间和简单所有权。动态扩容的双端队列需要处理旧缓冲区何时可回收；当前实现把容量之外的工作交给另一个结构，避免在此环上引入缓冲区迁移协议。

源码：bounded_queue.hpp:188-201；executor.hpp:1727。

07 pop: 多数情况只动尾部

算法摘写 / 保留关键操作顺序

```
b = bottom.load(rlx) - 1
bottom.store(b, rlx)
fence(sc)
t = top.load(rlx)
if t > b: bottom.store(b + 1, rlx); return null
value = slot[b & mask].load(rlx)
if t == b: resolve_last_item_with_top_CAS()
return value
```

Owner 先减 bottom，相当于声明“不再把最后一个槽暴露给新窃取者”。随后通过 seq_cst fence 再观察 top，区分三个情况： $t < b$ 仍有多个候选元素，Owner 取尾； $t == b$ 只剩一项，必须与窃取者竞争； $t > b$ 已空或被抢走，恢复 bottom 并返回空。

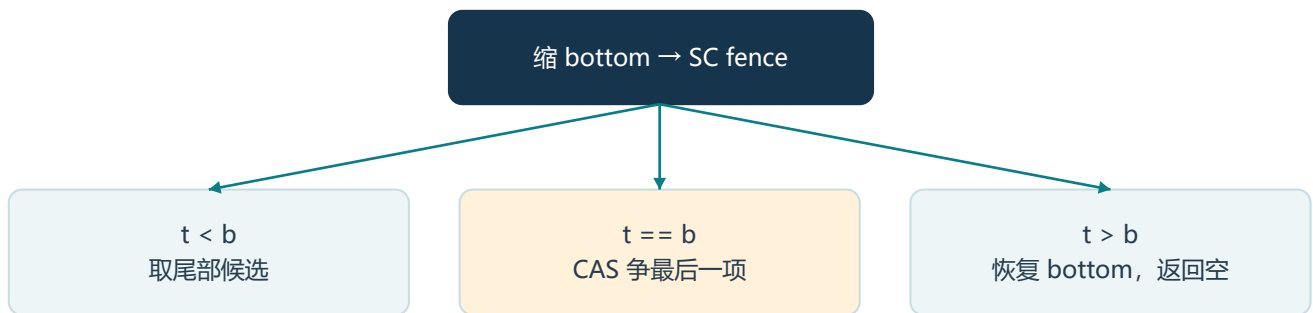


图 07: pop 的三种分支

多元素路径省去 Owner 对 top 的 CAS，使 Owner 与窃取者通常作用在两端。但“通常不同端”本身并不足以证明安全：当多个窃取者与 Owner 在接近空队列时交错，必须由这里和 steal 中的强顺序配合边界观察。

设计解释：SC fence 的角色是协调两个独立原子边界的观察，不是刷新槽位。仅因为 x86 上若干压力测试通过就将其降为 acquire fence，会改变算法的 C++ 层约束。任何弱化都需要针对完整 pop/steal 配对协议重新证明。

源码：bounded_queue.hpp:248-275。

08 最后一项：读到同一指针也只能一人赢



图示为 thief 赢的交错；Owner 赢时结果对称。

图 08：Owner 与 thief 争夺逻辑位置 20

初始 $top=20, bottom=21$ 。Owner 先令 $bottom=20$ ，读到 $top=20$ ；一个较早观察到 $bottom=21$ 的窃取者也可能持有槽 20 的指针。此时两者都尝试 $CAS(top, 20, 21)$ ，只有一个能成功。读取指针的次数可以是两次，取得执行权的次数只能是一次。

窃取者先赢。 它把 top 变为 21 并返回 Work。Owner 的强 CAS 失败，主动把返回值改为空，再将 $bottom$ 恢复为 21。稳定态重新满足 $top=bottom$ ，且 Owner 不会执行已被窃取的 Work。

Owner 先赢。 Owner 推进 top 并恢复 $bottom$ ，返回该指针；窃取者 CAS 失败，丢弃之前的推测读取。

Owner 已先缩尾而 thief 尚未取得有效边界。 thief 可能直接看到空，不必进入 CAS。

这里为什么使用强 CAS？最后一项 Owner 路径没有重试循环；若允许无竞争的伪失败，就会错误放弃唯一元素。steal 同样只尝试一次强 CAS，失败返回空交由调度器换目标。

不能在 CAS 成功前执行 callable、增加观察者统计或读取会随任务结束销毁的数据。此前的指针只是候选值。队列之外还必须确保同一 Work 没有被重复发布到另一处，否则这个 CAS 无法替整个任务系统去重。

源码：bounded_queue.hpp:258-267, 278-292。

09 steal: 一次尝试，而非阻塞出队

算法摘写 / 保留关键操作顺序

```
t = top.load(acq)
fence(sc)
b = bottom.load(acq)
if t >= b: return null
value = slot[t & mask].load(rlx)
if top.CAS_strong(t, t + 1, sc, rlx): return value
return null
```

一次成功窃取在 top CAS 处取得该逻辑头部的所有权。先读 top、再经 SC fence 读 bottom，与 Owner 的缩尾/读头顺序配对，处理接近空队列时的交错。bottom acquire 还承担观察已发布槽位内容的职责。

多个 thief 可以同时读到相同 top 和相同指针，随后争夺同一个 CAS。输者不在这个函数里重试。这样避免一个热点队列把空闲 Worker 困在无限内部重试中；外层执行器可以更换 victim，探索其他本地队列或共享栈。

返回空有两个含义：没有观察到可取元素，或观察到元素但争用失败。因此一次返回空不证明全局无任务，连续几次失败也不证明应该立即睡眠。执行器需要探索预算、休眠登记和登记后的队列复查。

与批量窃取相比，单项窃取减少一次迁移对 victim 的扰动，代码也更小；代价是短任务、高度不均匀负载下可能需要更多原子操作和跨核访问。当前实现没有 steal-half，也没有按任务预计耗时选择窃取粒度。

前提仍是一个 Owner；增加 thief 数量不会授权其他线程操作 bottom。CAS 仲裁的是逻辑位置，不是对象引用计数。

源码：bounded_queue.hpp:278-294；executor.hpp:1151-1184。

10 为什么槽位也必须是 atomic



atomic slot 防数据竞争；逻辑 top 的 CAS 防错误归属。

图 10：落后的窃取者与环槽复用

考虑容量 8。thief T0 保存逻辑 top=2 后暂停；其他 thief 推进 top，Owner 继续发布，物理槽 2 最终被逻辑位置 10 复用。T0 恢复后仍可能读 slot[2]，但读到的是另一轮的指针。

只要逻辑 top 未绕回，T0 对旧 top=2 的 CAS 必然失败，所以它不应执行误读的 Work。**但 CAS 失败不能消除此前非原子读取产生的数据竞争。**若 slot 是普通指针，T0 的读取与 Owner 对复用槽的写入没有必须存在的同步关系，C++ 层已经不合法。原子 slot 使推测读取本身合法，而 top CAS 排除错误归属。

这展示了两层职责：原子槽解决“允许并发读写这个内存位置”，逻辑索引解决“这个指针属于哪一轮出队”。不能因为 slot 是 atomic 就认为可执行任意读到的值，也不能因为最终 CAS 会失败就省去原子槽。

同理，CAS 失败路径不能解引用这个过期候选对象。它可能早已由正确的消费者执行并销毁。当前实现只是加载和丢弃指针值；真正使用对象发生在成功取得逻辑位置后。

设计边界：物理环回绕与逻辑索引回绕不同。后者在当前有符号索引实现中必须被工作负载寿命约束排除，不能拿“有版本索引”作为无限运行时间证明。

源码：bounded_queue.hpp:129-136,199,257,286。

11 批量 push 与溢出移交



先发布本地前缀；剩余部分的回调失败不会撤回前缀。

图 11：剩余两个本地槽，批量提交五项

批量 push 读取 bottom 和 top，计算本地可容纳数量 $k = \min(n, \max(0, \text{capacity} - (b - t)))$ 。它逐项填入前 k 个槽，再只做一次 bottom release store，发布整个前缀；剩余 $n - k$ 个元素交给 overflow 回调。当前批量实现读取 top 使用 relaxed，单项版本使用 acquire，不能把伪代码中的两者混为一谈。

好处是减少逐元素发布边界的成本；执行器随后调用 `notify_n(n)`，按批次通知等待者。通知数量是新增可运行工作的提示，并不建立“第 i 个 Worker 必须取得第 i 项”的对应关系。

异常边界也分为两段。填槽时迭代器到 `Work*` 的转换若抛出，bottom 尚未推进，已写槽位没有成为新发布的逻辑元素。发布前缀之后，若 overflow 回调抛出，已经公开的前缀不会回滚；调用方必须理解这种部分成功语义。

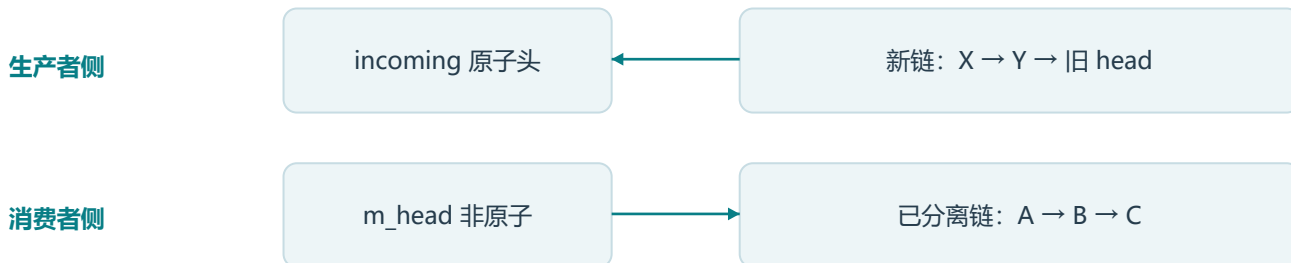
执行器传入的是有效 Work 指针范围，实际溢出回调把剩余元素串成共享链。库内部预期此调度路径不抛异常；泛型 `BoundedQueue` 自身的转换和用户回调仍有独立契约。

为何不等待本地空间？等待会让正在产生并行工作的 Worker 停下来，且可能让只有它能推进的计算无法继续。后备共享结构以额外争用换取提交路径的继续推进，但它不是限制内存增长的背压机制。

源码：bounded_queue.hpp:213-240；executor.hpp:1712-1725。

12 共享工作栈：三个字段，两个世界

```
state = [ LOCKED:1 | count:63 ]
```



只有 gate 持有者能操作 m_head；生产者持续向 incoming 发布。

图 12: incoming、私有 head 与 gate/count

m_incoming 是原子指针，所有生产者把新链压到这里。m_head 是非原子指针，仅取得消费者 gate 的线程访问；它保存先前从 incoming 整批摘下、尚未消费完的链。m_state 是原子 64 位字：最高位 LOCKED，低 63 位是数量。

这种分离让生产者不必等待消费者逐节点处理已经摘下的链。消费者把 incoming 一次 exchange 成空，随后多次 steal 都能从 m_head 取一个节点；直到 m_head 用尽才再搞一批。

与经典 Treiber 栈逐节点 CAS 出栈不同，当前结构用一个短消费者临界区保护 m_head，避免多个消费者对已分离链头并发解引用、修改和回收。生产者只把自己的尾节点连到观察到的 incoming 指针，不遍历已有链。

低位计数并不只代表“此刻 incoming 和 m_head 上能立即看到的节点数”。生产者先预留计数，再发布链，因此有一个计数已增加、节点尚未链接的窗口。这个设计直接影响 empty 与 steal 的语义，下一节逐步展开。

消费者失败返回空，不等待 gate 释放。尽管函数迅速返回，若 gate 持有者长期停顿，其他消费者不能从该分片取工作；所以不能由“使用原子、没有 mutex”推出消费者整体具有 lock-free 进展保证。

源码：shared_work_stack.hpp:113-118,201-246。

13 共享栈生产者：计数为什么先增加

算法摘写 / 保留关键操作顺序

```
state.fetch_add(n, rlx)    // 预留数量
head = incoming.load(rlx)
do:
    tail.next = head
while !incoming.CAS_weak(head, first, rel, rlx)
```



预留先行避免消费计数下溢；暂时取空是允许的结果。

图 13：计数可见、链尚未发布的窗口

若先发布链，再增加计数，一个极快的消费者可能先取走节点并减数量，导致低位下溢或错误空判断。当前实现反过来：先预留，再发布。代价是消费者可能观察到 $\text{count} > 0$ ，但 m_head 为空， exchange incoming 也取到空。这次 steal 会释放 gate 并返回空，计数保持不变，等待生产者完成发布。

release CAS 是链的发布点。所有链内 next 和 Work 相关写入必须先完成；消费者 exchange acquire 观察到这条链后才能使用其非原子字段。 count 的 relaxed 加法本身不发布这些内容。

CAS 失败会更新生产者手里的 head ，下一轮重新写尾节点 next ，再尝试。对于单项 push ， $\text{first}=\text{tail}=\text{work}$ ；对于批量 push ，内部节点已按迭代顺序串好，仅尾节点参与重试。

进展边界。 数量字段有 63 位，但仍要求总量不越界；增量不能侵入最高 gate 位。预留后生产者停顿，会使 empty 持续偏向“有工作”，从而让执行器继续探索。当前协议选择避免漏工作，接受这种暂时多探测。

源码：`shared_work_stack.hpp:121-197`。

14 共享栈消费者：gate 的取得与释放

算法摘写 / 保留关键操作顺序

```
s = state.load(rlx)
if locked(s) or count(s)==0: return null
s = state.fetch_or(LOCKED, acq)
if locked(s): return null      // 不能替赢家解锁
if count(s)==0: unlock_keep_count(); return null
if head==null: head = incoming.exchange(null, acq)
if head==null: unlock_keep_count(); return null
w = head; head = w.next; w.next = null
state.fetch_sub(LOCKED + 1, rel)
return w
```

取得 gate 使用 `fetch_or` 返回的旧值判胜负。两位消费者即使都在预检时看见未锁，也只有第一个 RMW 的旧值不含 `LOCKED`；第二个必须直接返回，不能清 gate，否则会把第一个仍在使用的私有 head 暴露给第三个消费者。

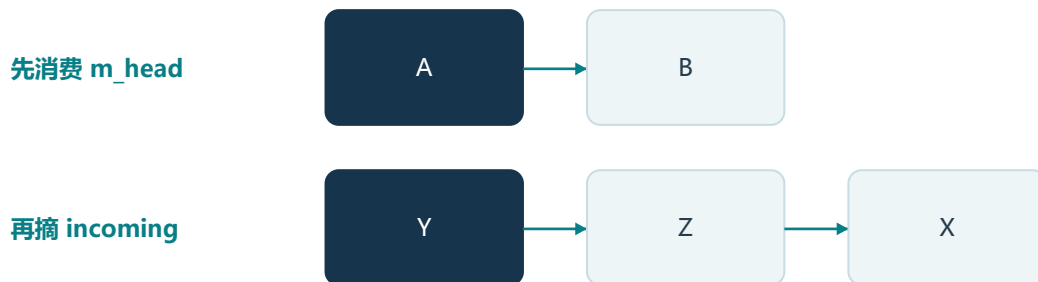
成功取走一个节点后，用一次 `fetch_sub(LOCKED+1)` 同时清最高位和减数量。这里不能保存进入时的数量、最后普通 store 回去：生产者可以在临界区中增加 count，旧快照覆盖会丢失这些并发增量。

空分离链路径使用 `fetch_and(SIZE_MASK)` 只清 gate，不减数量，因为预留中的生产者仍占一份计数。gate 的 release 与下一位消费者的 acquire 传递 `m_head` 及其链操作；incoming 的 release/acquire 则单独传递生产者发布。

顺序非常严格：读取 next、设置新 head、清空 `w.next` 全部必须先于放开 gate。return 后 Work 可能立即执行、转入另一等待链或销毁，容器不再拥有它。

源码：shared_work_stack.hpp:201-234。

15 共享栈到底按什么顺序取工作



结果：A、B、Y、Z、X。跨批次既非严格 FIFO，也非全局 LIFO。

图 15：已有私有链优先，新批次留在 incoming

假设 `m_head` 已保存 `A→B`。生产者随后依次推入单项 `X` 和批次 `[Y,Z]`，`incoming` 变为 `Y→Z→X`。消费者先返回 `A`、`B`，待私有链耗尽，再 `exchange incoming`，接着返回 `Y`、`Z`、`X`。

因此它既不是全局 FIFO，也不是对所有提交严格 LIFO。单项在 `incoming` 上有栈式倾向；同一批次内部保持构链顺序；已分离的旧链又先于后来到达的新链。再加上多个分片、多个 Worker 和本地接力，用户不能用提交时间推断最终开始或结束顺序。

为什么保留私有链？每次取工作都交换 `incoming`，会增加生产者与消费者对同一个缓存行的争用；分离一批后逐项消费，把后续消费主要留在 `gate` 与私有 `head` 上。代价是更多阶段状态，以及“计数非零但暂时取空”的接口语义。

单个 Work 指针多次入栈不是重复排队的合法方式：它只有一个 `next` 字段，重复链接会破坏结构。批次的首尾和元素数量必须匹配，尾节点的 `next` 由 `push` 接管。

设计边界：容器不提供公平等待、优先级或持久排队保证。若业务要求严格顺序，需要通过依赖边表达，不能依赖某次测试中恰好观察到的执行先后。

源码：`shared_work_stack.hpp:142-176,218-234`；`executor.hpp:1682`。

16 从提交到执行：三条快慢路径

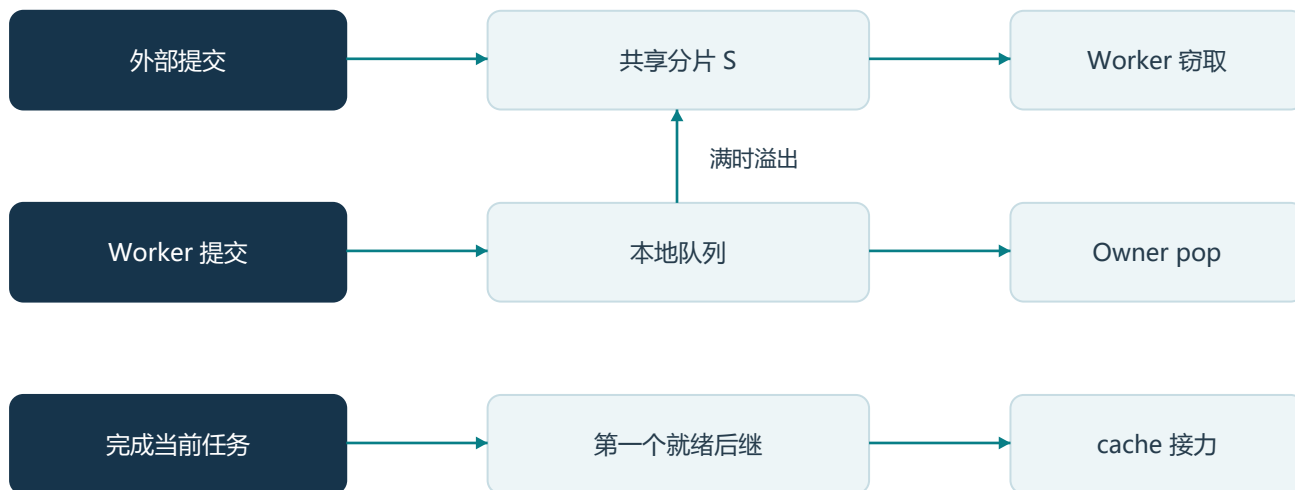


图 16：外部提交、本地提交和后继接力

外部线程提交时，执行器不能让它冒充本地队列 Owner，所以选择共享分片。Worker 内部提交优先写自己的有界队列，溢出再去共享分片。正在完成的任务若产生就绪后继，还可把其中一项放入 cache，完全不经过队列和 Notifier。

有 N 个 Worker 时，共享分片数 $S = \text{bit_width}(N) = \text{floor}(\log_2(N)) + 1$ ，窃取目标总数 $Q = N + S$ 。例如 $N = 8$ 时 $S = 4$ 、 $Q = 12$ 。它不是每 Worker 一个共享队列，也不是固定全局一个入口。

分片选择先将 Work 指针值乘以 64 位常数 11400714819323198485，再取与 S 相乘结果的高 64 位。这样把混合后的整数缩放到 $[0, S)$ ；批量提交依据第一项选择一个分片，整批一起进入。分散效果受分配地址和提交模式影响，并无逐项轮转或严格均衡保证。

`_schedule` 在发布队列之后调用 `notify`。通知提前于发布会让被唤醒 Worker 再次看到空队列，因此顺序不能交换。与此相对，cache 是当前线程已经取得的执行权传递，不需要用队列发布来让自己重新取得它。

数据寿命建立在更早一层：父 slot、执行引用、Topology 活跃计数必须先登记再调度。容器只负责转交可运行 Work，不替调用者补做这些生命周期操作。

源码：`executor.hpp:845-849,1682-1742,1847`。

17 窃取策略：记忆、随机化与退让



任何一次成功取到 Work，都回到执行；唤醒后重新搜索。

图 17: Worker 的工作搜索状态机

Worker 主循环先完成 cache 接力，再 pop 自己的尾部；只有没有本地工作时才进入 `_wait_for_work`。探索开始时尝试保存的 victim；成功后记住它，利用该位置近期仍可能有工作的局部性。失败后使用 Worker 私有 SplitMix64 选择下一个目标。

victim 空间同时包含 N 个本地队列和 S 个共享分片。初始 victim 为 $(worker_id+1)\%Q$ 。当前实现没有 NUMA 距离表、工作权重估计和任务优先级；随机索引也可能选回自己的本地队列，这仍是合法 steal 尝试，只是收益有限。

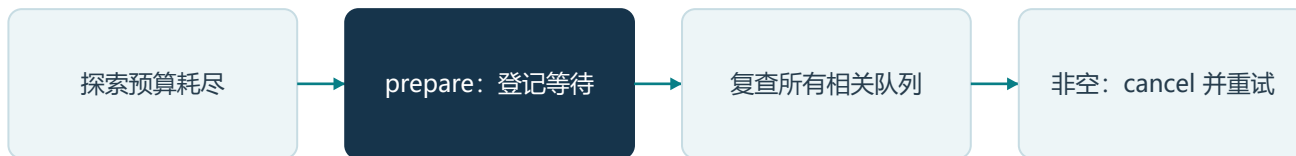
一次探索的偷取阈值为 $2*Q$ ，之后 yield 并增加退让轮次；最大退让轮次为 $64*bit_width(N)$ ，再进入准备休眠。代码用“超过阈值”判断，因此这些是控制预算的参数，不宜把它们直接当成精确失败调用次数。

这些预算折中是空闲延迟和 CPU 消耗：一直自旋可以迅速响应短间隔到达，但空闲时浪费核心；立即休眠节电，却增加唤醒成本。当前参数是启发式配置，不是关于最优吞吐的理论结论。

随机化降低所有空闲 Worker 同时冲向固定目标的倾向，却不保证某个任务在有界时间内被选中。加上本地 LIFO 和接力链，不能对外承诺 FIFO、公平性或实时截止时间。

源码：executor.hpp:1113-1190；random.hpp:109-165。

18 睡眠之前为什么必须再检查队列



全部为空且未终止才 commit; 终止出口也要 cancel。

图 18: 探索失败不等于可以安全睡眠

Worker 用尽探索预算后执行 `prepare_wait(id)`, 把自己登记为准备等待者; 接着扫描所有共享分片和其他 Worker 的本地队列。只要观察到一个非空位置, 就 `cancel_wait`, 记录这个 victim, 再回到探索。全部仍为空, 才检查终止位并 `commit_wait`。

登记与复查必须是这个顺序。若先判空再登记, 一个生产者可能恰好在两步之间发布任务, 发现没有等待者而不唤醒; Worker 随后登记并睡下, 任务就可能一直无人处理。复查让新发布工作有机会被自己看到; Notifier 协议则保证另一侧的通知不能同时被漏掉。

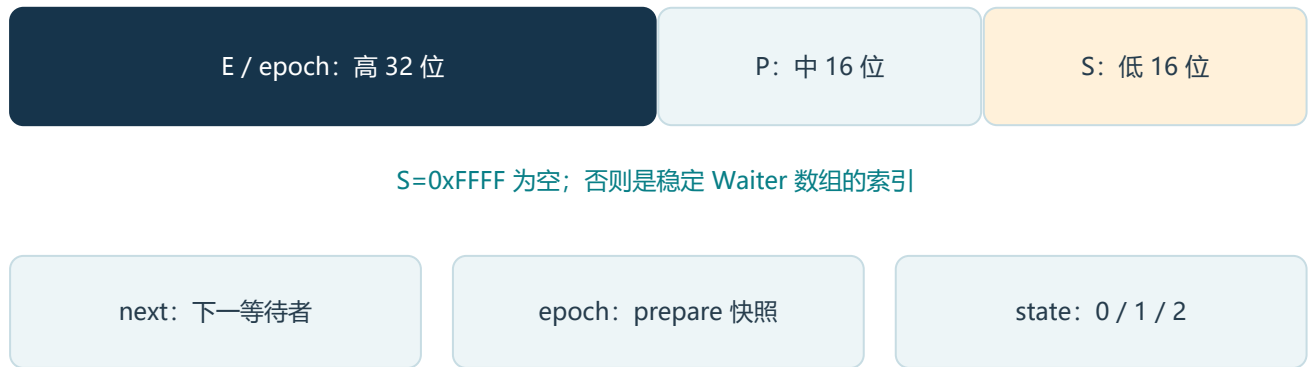
复查不扫描自己的本地队列。当前 Worker 是唯一 Owner, 进入此路径前已将它取空; 其他线程只能从中拿走工作, 不能向它 push。外部新增工作走共享层, 因此这项省略依赖于本地队列的单 Owner 约束。

共享栈 count 可以包含尚未发布的节点, 所以复查“非空”后重新 steal 仍可能失败; 这是保守重试, 不是协议损坏。Notifier 被唤醒后也总是重新搜索, 因为其他 Worker 可能已经拿走相关任务。

终止路径同样在登记后取消等待, 不能带着未完成的票据直接退出。Executor 析构先等待活跃拓扑结束, 再设置终止位、`notify_all` 并 `join`; 它不会替业务自动中断一个永不结束的 callable。

源码: `executor.hpp:1092-1110,1191-1227`。

19 Notifier: 把等待拆成票据与停泊



P 是预等待数量; S 不是数量, 而是已提交等待栈的头索引。

图 19: 64 位状态字及每 Worker 的 Waiter

全局状态字由高到低为 E:32 | P:16 | S:16。E 是推进中的 epoch; P 是已经 prepare、尚待处理的预等待数量; S 是等待栈头的 Worker 索引, 0xFFFF 为空栈。每个 Worker 对应稳定存储的 Waiter, 包含 next、prepare 快照和三态原子 state。

为什么既有 P 又有 S? 登记准备休眠的线程, 还必须回去检查业务队列, 不能立即把自己当成已经挂起的线程。P 表示这一段“可能即将睡”的窗口; S 记录已经完成提交、可被摘出唤醒的等待者。

为什么还要 E? 多个线程先后 prepare, 随后却以任意顺序 commit 或 cancel。E 为这些准备票据建立处理次序, 并让一个被通知提前消费的等待者知道“本次等待已经结束”, 而不是重复减少 P。

线程数量必须小于 65535: 索引哨兵和 P 位宽共同限制规模。构造函数检查 N 不为零且不超过此实现的容量。这个范围约束不能自动替代 epoch 长时间回绕的完整证明。

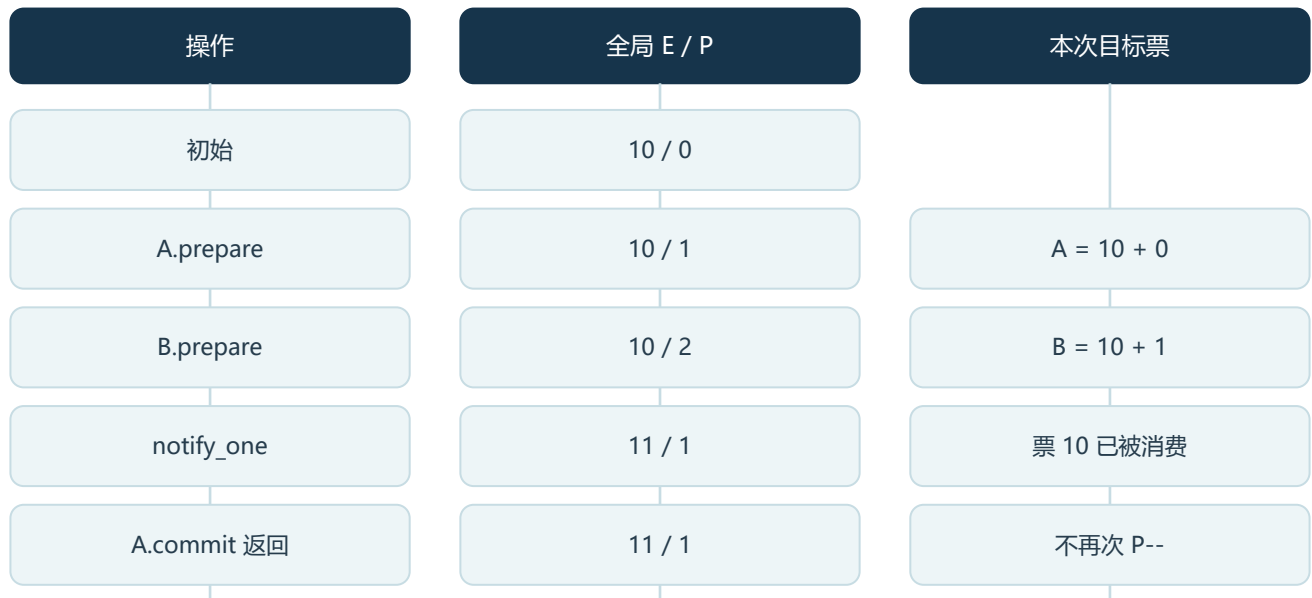
Waiter 按缓存行对齐, 以减少多个 Worker 写自身 state 时的伪共享。它是执行器生命周期内稳定的节点; 重用依靠票据和三态协议, 而不是每次睡眠重新分配节点。

源码: notifier.hpp:192-222,253-267; executor.hpp:833-842。

20 prepare: 怎样得到自己的等待票号

算法摘写 / 保留关键操作顺序

```
snapshot = global_state.fetch_add(P_INC, r1x)
waiter.epoch = snapshot
fence(sc)
target_epoch = E(snapshot) + P(snapshot)
```



B 的票 11 仍待处理；Worker 索引和等待票号不是同一个数。

图 20：两个预等待者分别取得票据 10 和 11

假设当前 $E=10$ 、 $P=0$ 。A prepare 得到旧快照 $(10,0)$ ，自己的目标票为 10，P 变成 1；B 随后得到 $(10,1)$ ，目标票为 11，P 变成 2。代码保存完整快照，在 commit/cancel 中用掩码和移位计算目标 epoch。

这个票号表示本次 prepare 的处理位置，不是 Worker 固定编号。后来的 Worker 即便先到 commit，也必须等待前面的票通过 commit、cancel 或 notify 推进。这样同一个计数 P 不会被任意线程无序重复消费。

prepare 的 RMW 使用 relaxed，之后单独执行 SC fence。它关注的是“已登记等待”和后续业务队列检查之间的全局观察次序；Work 内容的发布仍由队列自己的 release/acquire 完成。

通知可能在 A 尚未 commit 时到达。若通知消费一个预等待票，E 增到 11、P 减到 1。A 再处理票 10 时发现全局 epoch 已领先，直接返回，不需要真的入栈或停泊；B 的票 11 仍待处理。

必须保持每次 prepare 最终恰好进入一次 commit 或 cancel 路径。不能重复 commit、漏 cancel，或把同一 Worker 的 Waiter 同时用于两次等待。

源码：notifier.hpp:284-318,362-381。

21 commit 与 cancel：同一张票的两种结局

全局 E 与目标票	commit_wait	cancel_wait
E 落后	yield, 重新读取	yield, 重新读取
E 已领先	已被处理, 直接返回	已被处理, 直接返回
E 恰好相等	P--, E++, 压入等待栈	P--, E++, 保持栈不变

commit 先把本 Worker 的 state 重置为 NotSignaled, 再准备 next 指向旧栈头, 用 release CAS 发布新的全局栈头, 同时推进 E、减少 P。成功后进入 _park。发布前写 next 和 state, 通知方通过状态字 acquire 取得这些内容。

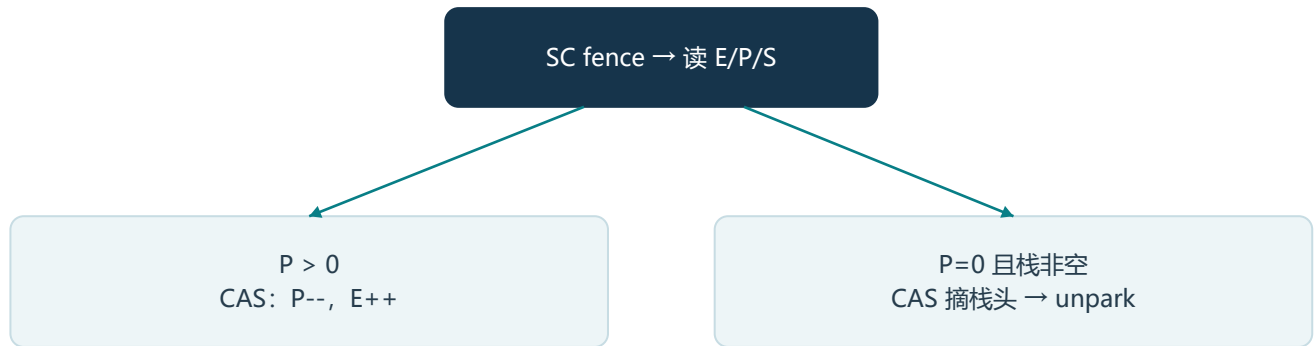
cancel 不会入栈; 它只在轮到自己时用 relaxed CAS 消费票据。取消也必须推进 E: 若只 P--, 下一张票永远等不到自己的 epoch, 整个等待系统就可能卡在过期票号上。

当 notify 已经替该票推进 E, commit/cancel 直接返回。这解释了为什么不能无条件 P--: 同一张票可能由等待者自己处理, 也可能由通知者处理; epoch 判断避免二者重复扣减。

回绕条件。 E 只有 32 位, 代码用差值转有符号数判断先后, 本质需要活跃比较距离处于半个序号空间之内。一个已被提前通知、却极长时间未恢复的线程, 不能只用“最多 65534 个 Worker”就证明其快照年龄有界。本文把这列为需要形式化审查的序号前提, 而不把它写成已经复现的故障。

源码: `notifier.hpp:302-400`。

22 notify: 先消费预等待, 再摘等待栈



预等待者自行发现票已被消费；已入栈者通过三态协议停止等待。

图 22: 通知的两条分支

`notify_one` 先做 SC fence, 再 acquire 读取全局状态。若 $P > 0$, 优先用 CAS 做 $P--$ 、 $E++$, 由一位预等待者自行发现其票已被消费；这条路径不调用操作系统唤醒。若 $P=0$ 且栈非空, 读取栈头 next, 用 CAS 摘一项, 再 `_unpark`。

`notify_all` 用一次 CAS 清空 P 和等待栈, 把 E 增加原 P 数量；然后沿摘下的栈逐个唤醒。`notify_n` 对预等待票可按 $\min(n, P)$ 批量消费, 已入栈节点则逐项摘出； n 足够大时直接转 `notify_all`。

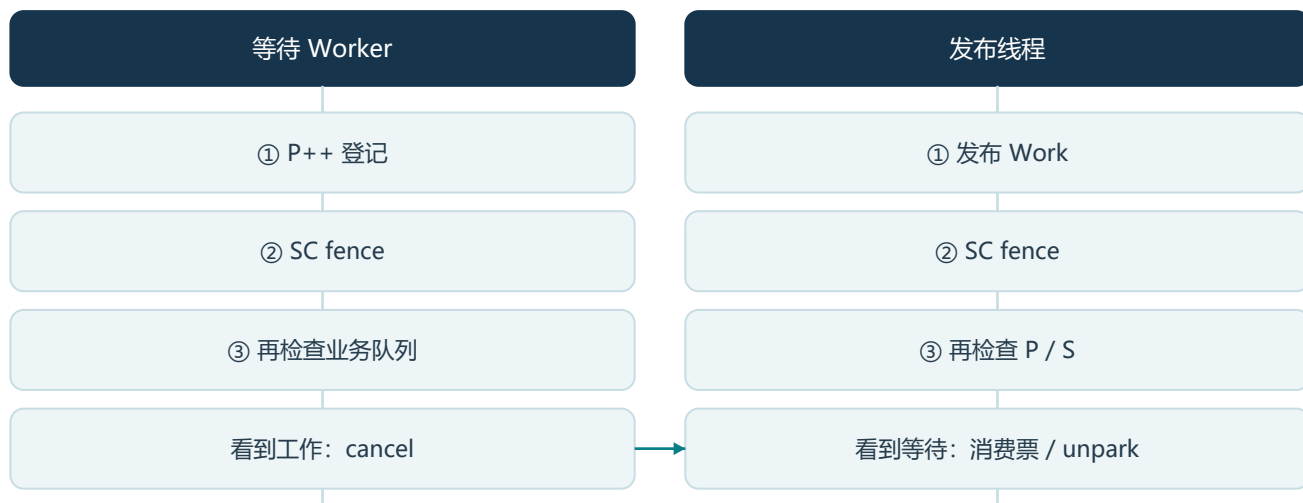
这套协议让“新工作到达”既能阻止尚未停泊的线程睡下, 也能唤醒已经入栈的线程。 P 优先的取舍是减少不必要的底层唤醒, 但它不提供按 Worker 编号公平服务的保证。

实际内存序。 当前 `notify` 的成功状态 CAS 使用 `acquire`; `commit` 入栈的 CAS 使用 `release`。不能为了图解整齐把它写成统一 `acq_rel`。等待栈可见性需要结合原子状态字的 RMW 链与 `release sequence` 分析, 而不能拿一个孤立的 `acquire CAS` 当作对后续线程的普通 `release`。

`notify` 只处理等待协议, 不携带任务指针；真正的数据可见性还要由随后成功取工作时的队列协议建立。

源码: `notifier.hpp:411-544`。

23 丢唤醒：两个线程不能同时“没看见”



危险组合：两边同时漏见。正确性需结合原子访问与 fence 完整分析。

图 23：登记/复查与发布/通知之间的交叉约束

等待方顺序是“登记 P → SC fence → 检查队列”；生产方顺序是“发布工作 → SC fence → 检查等待状态”。错误结局需要同时发生两件事：等待方认为队列空，生产方认为无人等待。协议利用两侧 fence 以及对应原子读写约束，排除这一双重漏见的危险组合。

可分成两种直观情况理解。若工作发布足够早，登记后的队列复查会让 Worker 取消等待并重试取工作；若发布较晚，生产者会遇到已登记的预等待者或已入栈的等待者，消费其票或将其唤醒。通知发生在 commit 前并不等于丢失，因为票据仍记录本次等待已经被处理。

这里是协议级解释，不是完整形式化证明。证明还必须覆盖：empty 的快照语义、共享栈预留窗口、epoch 回绕前提、全局状态 CAS 的失败路径，以及所有 prepare 出口。只画两根箭头并不能自动证明整份程序。

还要区分安全性与活性。协议避免漏唤醒，不代表持有资源的线程一定继续运行，也不解决用户 callable 中的永久阻塞。调度器无法凭通知机制为一个永不归还的信号量配额创造资源。

源码：notifier.hpp:284-293,411-415; executor.hpp:1191-1227,1712-1742。

24 park/unpark: 封住最后一个睡眠窗口

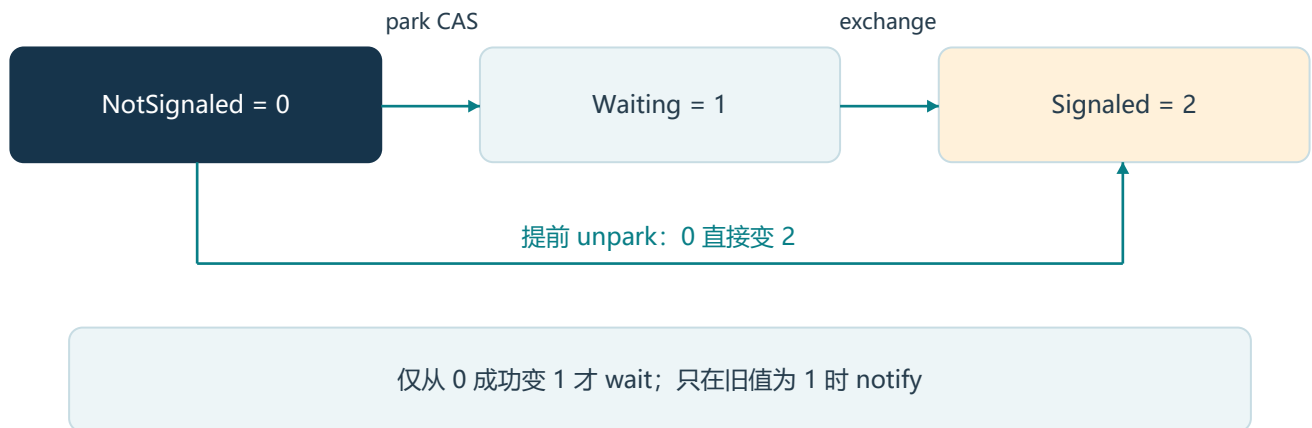


图 24: NotSignaled、Waiting、Signaled 三态

入栈成功后到真正执行 atomic wait 之间，仍然可能来一次通知。`_park` 先尝试把 state 从 NotSignaled 改为 Waiting；成功才执行 `state.wait(Waiting)`。`_unpark` 则 exchange 成 Signaled，只有旧值是 Waiting 才调用 `notify_one`。

通知先发生。 unpark 把 NotSignaled 改成 Signaled；随后 park 的 CAS 失败，线程根本不睡。**park 已置 Waiting、还没调用 wait。** unpark 改值并通知；随后 atomic wait 检查到值不是 Waiting，会返回。**已经等待。** 改值加 notify 使其醒来，之后重新找工作。

所以不能把 unpark 简化成单独 `notify_one()` 而不改变状态值。原子等待是围绕“值是否仍等于 expected”工作的；notify 不是永久存储的信号量令牌。

这里 state 的操作使用 relaxed，因为这个三态机只决定是否等待，不用于发布 Work 内容。真正的工作数据同步来自队列；等待链 next 的发布来自前面的状态字协议。

unpark 遍历链时，先读 next，再 exchange 当前 Waiter 为 Signaled。唤醒之后，所属 Worker 可能快速开始新一轮并复用 Waiter；先缓存 next 避免随后读取已经被重写的链接。C++ atomic wait 的底层实现因平台而异，不能把本库在 Windows 上的行为一律称为 futex。

源码: `notifier.hpp:585-611`。

25 图边如何保存，为什么分前后两段

`m_edges`: 同一个连续 vector, `m_num_successors=3`



连接 $A \rightarrow B$ 要更新两端；第二端追加失败时撤回第一端。

图 25：一个 vector 同时保存后继与前驱

Work 用一个 `vector<Work*> m_edges` 保存两段：前缀 $[0, m_num_successors)$ 是后继，余下是前驱。执行完成最常遍历后继前缀；初始化 join weight 则遍历前驱后缀。分区避免为每个节点再增加一个独立容器对象。

增加边 $A \rightarrow B$ 时，先将 B 放到 A 的后继区，再将 A 追加到 B 的前驱区。如果后一次分配抛异常，代码删除刚插入的 A 后继，恢复双向一致性。删除通过末项交换压缩，因而不能把边的物理存储顺序当成永久稳定标识。

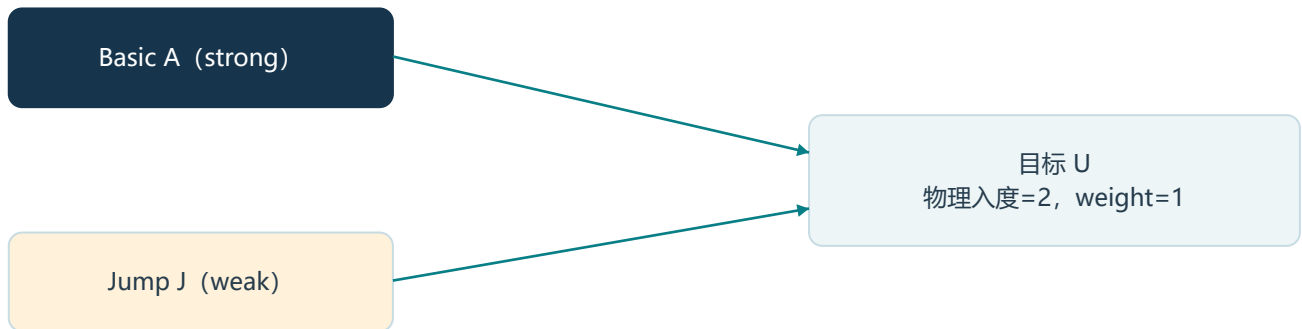
启用检查的连接路径拒绝空目标、跨 Graph、重复边，并搜索是否形成没有 Jump 节点的严格环。DFS 遇到 Jump 类型会截断严格路径；允许 Jump 相关循环不等于证明这个循环一定终止或不会重复激活同一个运行中节点。

图构建与执行不是任意并发的。运行中 Work 的父指针、计数、边前缀都可能被执行器使用或调整，调用者不能同时修改同一个 Graph。批量连接若由多次单边插入构成，也不应推断它具有整个批次的事务回滚保证。

设计取舍：物理边是便于遍历与校验的双向结构；运行期就绪仍由计数推进，不会在每次完成时重新全图拓扑排序。

源码：work.hpp:1250-1396；executor.hpp:1331-1338。

26 初始化图：物理入度不等于 join weight



即使只保留 $J \rightarrow U$, $\text{weight}=0$ 的 U 也不是物理源点。

图 26：弱前驱不计 join，却仍使节点不是源点

`_set_up_graph` 为节点绑定父 Work 和 Topology，清理本轮异常标记，计算 strong 前驱数量并编码到 Properties 的低位，然后初始化非源节点的 join 计数。真正的源点依据**物理前驱数**为零判断，并交换聚集到 Graph 数组前缀。

例如 U 有一条来自 J 的弱边，则 U 的 join weight 可以为零，但物理入度为一。 U 不应在图开始时自动执行，因为它需要等待 J 的控制转移。若把 “join weight=0” 直接当成源点条件，就会让控制目标在跳转前提前运行。

普通 Basic、Branch、Runtime、Module 等图节点是 strong 前驱；Jump/MultiJump 不以普通 join 方式传播。Branch 仍然是 strong，因此一个带多个 strong 前驱的汇合节点必须收齐其协议要求的到达；不能把互斥分支的未选边自动当作已完成。

初始化会重排源点前缀，不保留全局插入顺序。没有物理源点时，模块路径会结束而不会凭空寻找一个循环入口；一个闭合的 Jump 环也需要正确设计激活入口。

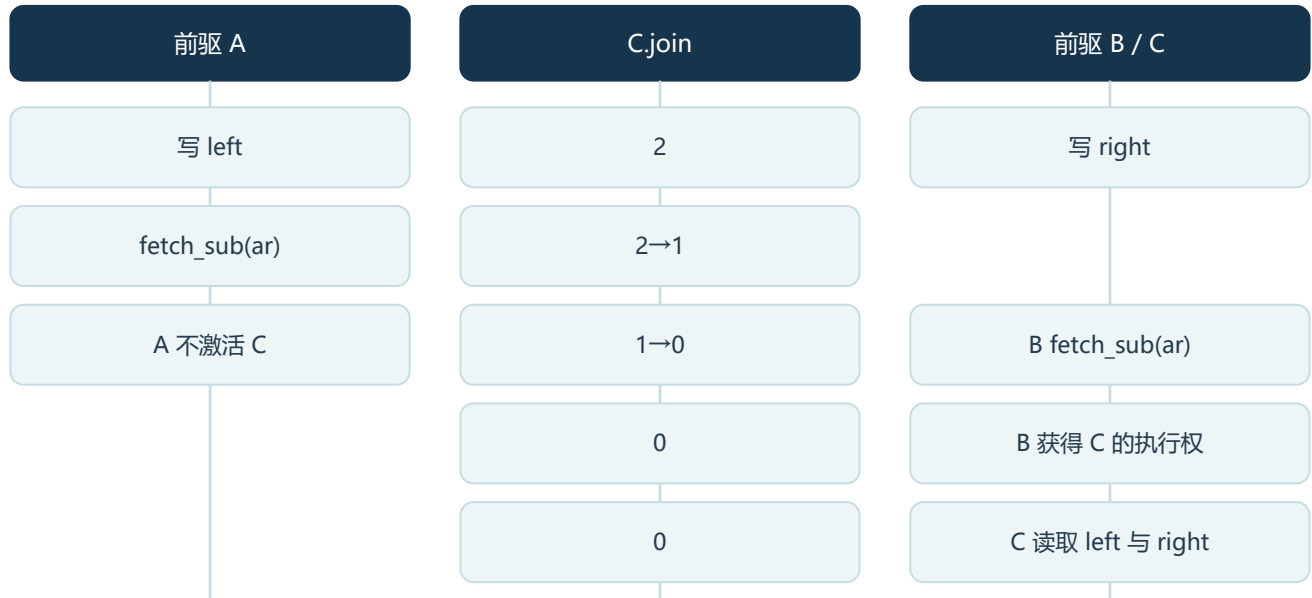
重复运行时 `_reset_graph_join_counters` 修复非源点剩余计数，适用于前一轮已经静止的边界。它与节点完成时的增量恢复用途不同，不能在运行中随意调用来“修正”计数。

源码：executor.hpp:1229-1282；work.hpp:656-683；work_invokers.hpp:1482。

27 join: 最后一个到达者取得后继

算法摘写 / 保留关键操作顺序

```
// 对一个有 k 个 strong 前驱的后继 C
C.join = k
on_predecessor_finish:
  old = C.join.fetch_sub(1, acq_rel)
  if old == 1: schedule_or_cache(C)
```



A 与 B 之间仍可并行；它们不得无同步地写同一普通对象。

图 27：两个前驱的写入经 join 到达 C

若 A、B 都是 C 的 strong 前驱，初值为 2。先到者将 2 减成 1，不调度；后到者将 1 减成 0，获得唯一的普通激活权。判断必须使用 RMW 返回的旧值，不能先 load 判断“快归零了”再单独减。

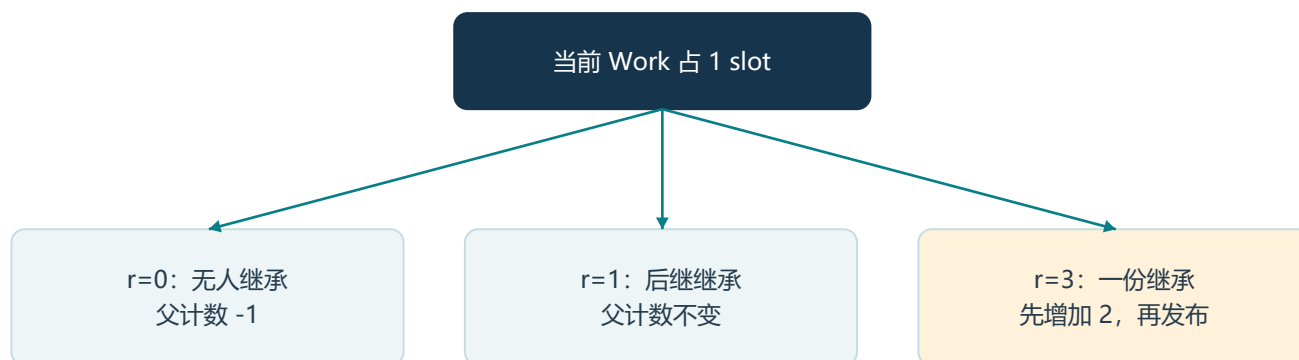
acq_rel 链不只用于计数。前驱在减计数之前的计算写入，经 release 发布；最后到达者通过 acquire/RMW 链取得先到达者的发布，再通过队列或当前线程接力让后继观察这些数据。在没有额外并发写的前提下，依赖图可以建立用户数据所需的先行发生关系。

这不意味着前驱间可同时写同一非原子变量。A 与 B 在汇合前仍可能并行；join 只把二者排在 C 前面，不为二者互斥。

节点完成后，先 fetch_add(join_weight, relaxed) 恢复下一次激活所需基数，再传播后继。不能直接 store 静态权重：有循环控制转移时，下一轮的合法到达可能已提前减过计数，store 会覆盖它们。增量恢复可保留这些到达，但也不自动允许同一 Work 任意重入。

源码：executor.hpp:1286-1334。

28 父 slot：完成计数究竟在数什么



计数代表尚未结束的活动责任，不是整张图剩余的节点数。

图 28：一个活动 slot 如何分裂、继承和归还

父 Work 的 join 计数不等于“图中尚未执行的节点数”。它统计当前仍未结束的活动执行责任，本文称为 slot。发布 n 个源节点前，父计数建立 n 份责任；后续线性链可以让责任直接从当前节点传给一个就绪后继。

一个节点完成时，若没有就绪后继，归还自己的一份 slot；若恰有一个，后继继承，父计数不变；若有 r 个就绪后继，其中一个继承原 slot，其余 r-1 个需要先增加父计数，再发布到队列。

守恒规则：每个已发布或正在接力的活动分支，必须由一份父 slot 覆盖；产生额外分支先加计数，责任无继承者才减计数。仅因某个后继还在等其他前驱，不需要给这个未就绪节点额外占 slot；仍在运行的前驱责任覆盖未来的推进。

若先发布后加计数，别的 Worker 可能极快执行完后继，将父计数减到零并恢复或销毁父任务，而提交方还在修改父状态。这是顺序错误，不能靠把加法改成更强内存序补救。

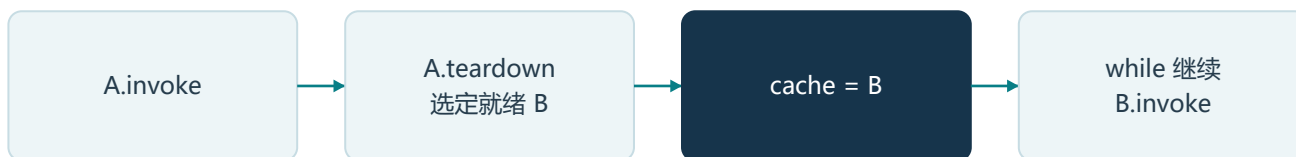
`_schedule_parent` 先保存 `PREEMPTED`，再 `acq_rel` 减父计数；只有旧值为 1 且父处于挂起阶段，才让父进入 `cache`。它在减计数后避免继续读取可能已转交所有权的父字段。

源码：`executor.hpp:1317-1364,1745-1755`。

29 cache 接力：为什么线性链不必反复入队

算法摘写 / 保留关键操作顺序

```
invoke_chain(w):
    do:
        cache = null
        w.invoke(worker, executor, cache)
        w = cache
    while w != null
```



一个直接接力位置；其他就绪节点先增加 slot 再进入队列。

图 29: A 完成后把 B 交给当前 Worker 直接执行

cache 是 `_invoke` 的局部 Work 指针，不是全局缓存表。tear_down 遍历后继时，把第一个已就绪节点写进去；其他就绪节点聚集后批量进入本地队列。调用返回后 while 循环直接执行 cache，因此长的普通依赖链不会每边都递归调用一层 C++ 函数。

它同时节省本地 push/pop、通知和父 slot 的一减一加，并保留当前核心上的数据局部性。取得 cache 的前提是依赖 CAS/RMW 或父完成协议已把唯一执行权交给当前路径；它不是绕过依赖判断的快捷开关。

`_schedule_parent` 恢复父任务时，如果 cache 已有别的 Work，会先把原 cache 排入本地队列，再把父放进 cache。一次调用链最终只有一个直接接力位置，其他工作都需要正常发布。

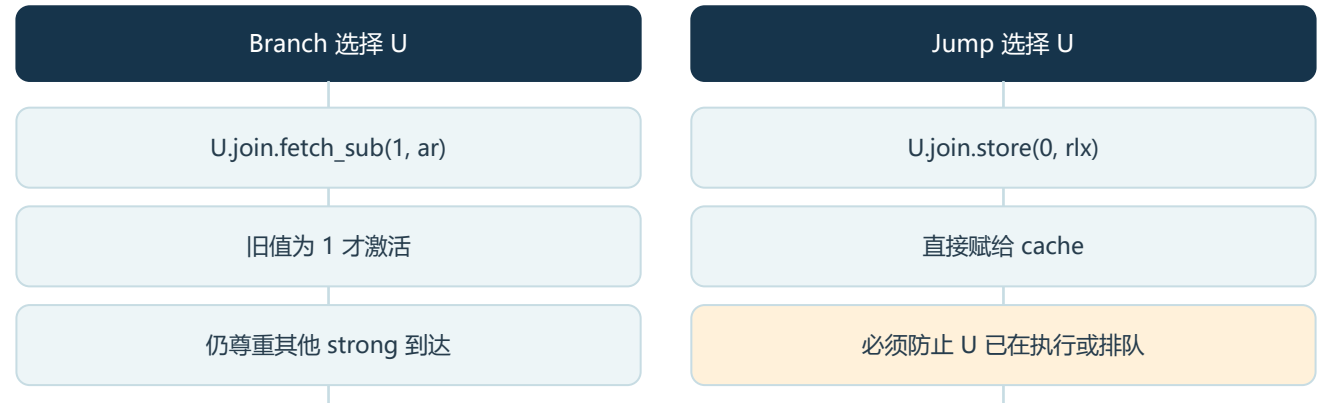
代价是公平性：一个不断生成单后继的长链可能长时间占住该 Worker；共享任务仍需其他 Worker 或当前链结束后才能处理。本实现没有按接力长度强制轮换的时间片，也不抢占已经运行的用户函数。

跨 Executor 的异步后继不能进入当前执行器的 cache；它必须发布到自己归属执行器的共享入口，确保运行线程与资源上下文一致。

源码：executor.hpp:1334-1364,1653-1663,1745-1755,1847-1853。

30 Branch 与 Jump 是两种执行协议

机制	对选中目标做什么	适合表达的逻辑
Branch	对目标 join 做 fetch_sub, 旧值 1 才激活	条件性 strong 到达
MultiBranch	对多个选中目标分别到达	多目标条件释放
Jump	目标 join 置 0, 直接交给 cache	显式控制转移
MultiJump	每个目标 join 置 0, 额外目标新建 slot	多目标强制激活



Branch 是条件性到达; Jump 是显式控制转移。

图 30: 条件到达与强制激活的差异

Branch 不是“无视其余依赖直接执行目标”。例如 C 还需要 A 的 strong 到达, Branch 选择 C 只完成其中一份。反过来, 两个互斥 Branch 都指向 C, 如果本轮只可能有一个选择 C, 静态权重却是二, 就不能期待 C 自动汇合。

Jump 明确绕过目标的普通 join 屏障。实现先将目标 join 置零, 使其处于类似普通激活完成后的计数状态, 然后安排执行。目标完成时仍会恢复自己的静态权重。

所以 Jump 是需要约束的控制流工具: 跳到仍在运行、已在队列中或还会被另一条普通到达路径激活的 Work, 可能造成重复执行和计数破坏。拓扑允许带 Jump 的环, 只说明静态建图规则允许, 不证明动态执行安全。

无目标或出现异常时, 当前节点不继续传播, 归还父 slot。多目标路径需要为额外目标增加父计数, 且必须先加后发布。

源码: executor.hpp:1367-1559; work_invokers.hpp:534-765。

31 循环与重复运行：两个不同层次



运行中增量恢复与静止边界重置具有不同的前提。

图 31：图内 `Jump` 回边与模块轮次边界

图内循环通过 `Jump/MultiJump` 将控制转回已有节点。每个节点的完成路径用 `fetch_add(weight)` 恢复其 `join` 基数，再推进后继；这种增量恢复允许保留协议上已到达的下一轮信号。图内循环期间，并没有一个全图静止点供任意重置所有计数。

模块重复运行则不同：本轮活动 `slot` 全部归零后，模块 `Work` 被恢复，检查停止、异常和 `predicate`；若继续，再修复非源节点 `join`，建立源节点计数，启动下一轮。这里轮次边界已静止，所以可以用 `store` 重置需要修复的计数。

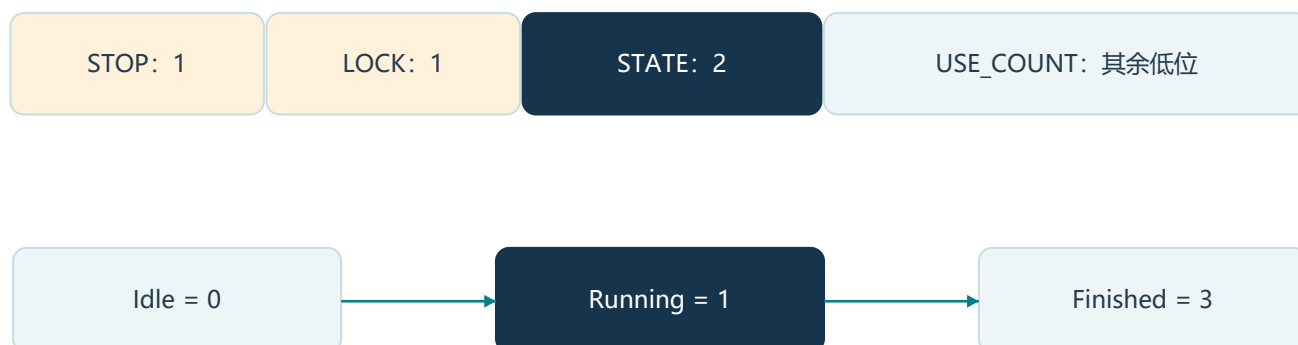
`predicate` 在每轮启动前判断，返回 `true` 表示停止。按次数运行通过一个递减计数的谓词实现，因此运行次数为零时不启动第一轮；`callback` 在模块结束路径调用。它们的抛异常行为由 `Work` 的异常处理辅助函数接管。

为什么需要轮次修复？条件路径可能让某些节点本轮只收到了部分 `strong` 到达，却没有执行自身 `teardown` 来恢复基数。若不在新一轮启动前重置，会把上一轮残留的计数带入下一轮。

设计前提：同一 `Graph` 的可变运行状态必须由一个运行实例独占。共享一个图对象的不同句柄不等于获得了多份 `join` 计数；并发提交同一个借用图需要由调用者排除。

源码：`executor.hpp:1269-1282,1291-1308`；`work_invokers.hpp:974-1025,1470-1510`。

32 异步状态：一个控制字里的四类信息



布局按 `size_t` 推导；Finished 后引用计数可以仍然大于零。

图 32: Topology 控制字与生命周期

Topology 的原子控制字把 `STOP_REQUESTED`、`LOCKED`、两位状态和强引用计数打包在一个 `size_t` 中。状态编码是 `Idle=0`、`Running=1`、`Finished=3`；引用计数占低位。位宽按目标平台的 `size_t` 推导，不能把整个布局固定写死成 64 位 ABI。

组合控制字的关键好处是：完成线程能用一次 CAS 同时检查“仍是 Running 且当前没有依赖登记锁”，再发布 Finished；不需要先独立读锁，再修改另一个状态原子，从而留下登记与完成交错的窗口。

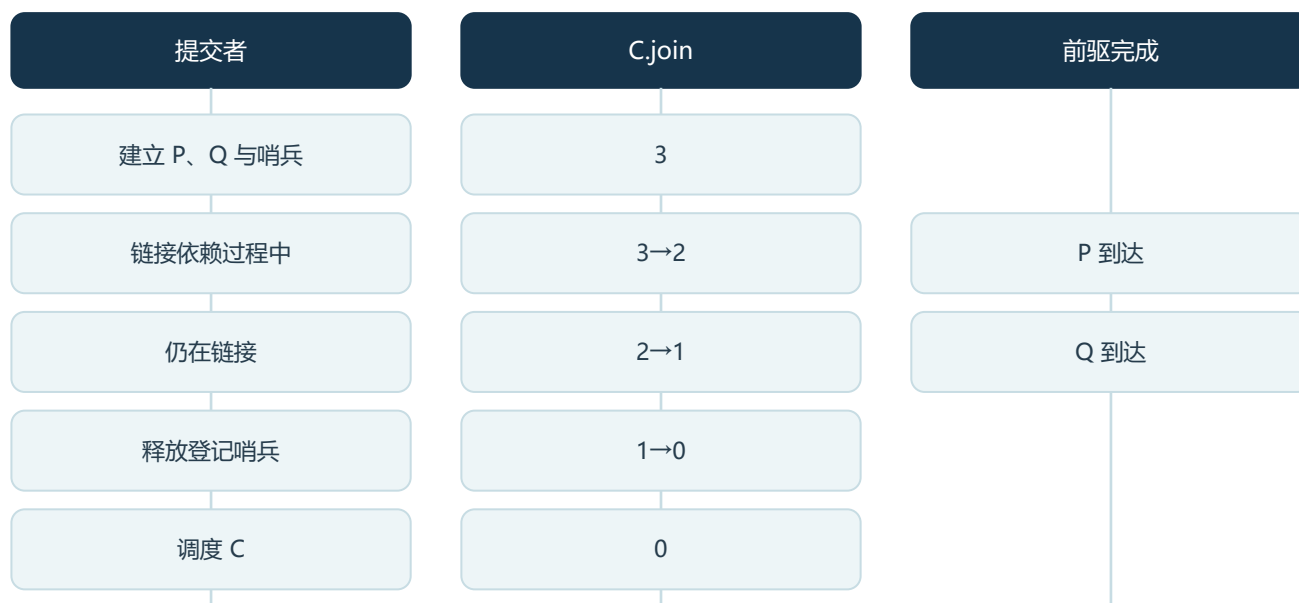
`AsyncTask::start` 先验证依赖，随后在 Idle 状态争取 LOCKED。赢家建立前驱强引用、join 和执行引用，发布 Running 后解锁；竞争的第二次 start 最终观察到非 Idle，抛出只能启动一次的错误。把 `AsyncTask` 转成 `Future` 本身不启动任务。

Executor 直接创建的异步任务没有对外可竞争的 Idle 启动阶段，其内部 `_launch_async` 可在独占初始化阶段用 relaxed 写 Running，最后依靠队列发布。不能把这一初始化捷径照搬到公开的 `AsyncTask::start`。

Finished 只代表本次执行结果已发布，不要求引用计数为零。外部 `Future`、`AsyncTask` 以及后继保留的前驱引用，都可以让一个 Finished Work 继续存活。

源码：topology.hpp:63-140；async_task.hpp:337-446；executor.hpp:881-945。

33 异步依赖的 +1 哨兵：封住提前启动



谁执行唯一的 $1 \rightarrow 0$ 转换，谁负责发布 C。

图 33：两个前驱加一份登记哨兵

创建依赖 P、Q 的异步任务 C 时，不能简单地先写 $\text{join}=2$ ，再逐个登记。若 P、Q 在登记期间快速完成，C 的计数可能提前归零并运行，而提交线程还在建立边或访问 C。当前实现设置 $\text{join}=\text{+1}$ ，额外的一份由提交线程自己持有。

每个已完成前驱，或随后完成的已登记前驱，消耗一份计数。只要提交哨兵尚未释放， join 至少为 1，任何前驱都不能把它从 1 减到 0。全部链接工作结束后，提交线程执行一次 `acq_rel fetch_sub`，释放哨兵。

若此时旧值是 1，所有前驱已经到达，提交线程负责调度 C；否则最后完成的前驱以后会成为旧值 1 的减计数者。这样“负责调度的人”可以不同，但零转换始终唯一。

空依赖句柄会被过滤。非空 Idle 依赖被拒绝，避免等待一个没有启动的前驱；AsyncTask 还明确拒绝依赖自己。重复给出同一个已启动前驱时，登记和计数按相同次数处理，不应凭直觉擅自把其计数当作已去重。

哨兵解决登记过程的早完成竞态，不解决内存分配失败回滚。链接过程中若 vector 扩容抛异常，必须另行审查计数、锁和引用清理；当前快照仍有未完备路径，见第 58 节。

源码：executor.hpp:891-937；async_task.hpp:357-435。

34 动态登记与前驱完成如何互斥



同一状态字 CAS 排除“检查后完成，再追加无人通知的边”。

图 34：登记先赢与完成先赢，两条都恰好到达一次

登记 C 到前驱 P 时，先 acquire 读 P 的控制字。若 P 已 Finished，直接对 C 的 join 减一，不再向 P 追加边；结果已完成，没必要让 P 再遍历一条新后继。

若 P 尚未完成，登记方尝试在不含 LOCKED 的状态上 CAS 加锁。成功后，将 C 放入 P 的后继前缀，再 release 清锁。P 的完成 CAS 只允许从未锁定的 Running 转为 Finished，所以它必须等此次登记结束，随后才能冻结并遍历后继前缀。

反过来，若 P 先成功发布 Finished，登记方的旧状态 CAS 会失败并读到 Finished，转为直接减 C 的计数。它不会把 C 追加到一份已经冻结、甚至即将结束遍历的后继表。

这个协议把“是否已经完成”与“是否还能追加后继”合在同一个原子裁决中。只用 `if (!finished) push_back()` 会出现检查后即完成的经典竞态：完成线程没见到新边，新边又永远等不到通知。

寿命也必须独立覆盖：C 的边后缀保存并持有 P 的强引用，确保登记和后续使用期间 P 不被释放；P 的后继指针本身不对 C 增加强引用，C 由执行期引用和启动协议保持存活。

源码：executor.hpp:1582-1665；async_task.hpp:403-410。

35 Finished 发布之后，收尾仍未全部结束

算法摘写 / 保留关键操作顺序

```
CAS(control, Running_without_LOCKED, Finished, acq_rel)
control.notify_all()
for each frozen successor:
    if successor.join.fetch_sub(1, acq_rel)==1:
        publish_to_its_executor_or_cache()
release_execution_reference()
return_parent_slot_or_decrement_active_topologies()
```

结果和异常在发布 Finished 前写入；等待线程 acquire 观察到 Finished 后，可以通过 Future 读取。发布后动态登记不再修改当前 Work 的后继前缀，完成线程便可无锁遍历这一冻结范围。

同 Executor 后继可以用 cache 或当前 Worker 的本地队列；不同 Executor 后继进入它自己的外部调度入口。依赖就绪与执行器归属是两项检查，跨池依赖不能把后继随意留在当前池运行。

完成标志会早于“遍历完全部后继、释放执行引用、活跃计数减一”。因此 future.wait() 返回只承诺该 Future 对应结果已经完成，不能据此推断整个执行器所有后续工作都已结束。全局收尾由 wait_for_all() 的活跃 Topology 计数覆盖。

释放执行引用可能立刻销毁当前 Work，所以代码在此前缓存 parent 和 topology 等必要值；回收之后不再访问 w，最后只归还父 slot 或减少执行器的顶层活动计数。

设计解释：把结果可观察时间与整段收尾分开，可以让后续观察者早一点继续，但要求实现严格区分已完成对象与已回收对象。

源码：executor.hpp:1617-1678,1832-1840；work.hpp:616-637。

36 引用释放为什么采用迭代回收



pending 借用零引用 Work 的 parent；不递归扩张 C++ 调用栈。

图 36：后继持有前驱，零引用链使用 pending 栈回收

异步 C 依赖 P 时，C 的前驱后继持有 P 的强引用。即使用户已经释放 P 的 Future，C 仍可能保留它。任务完成时释放的是本次执行引用；真正的最后一个引用释放者才进入 `_destroy_async`。

长依赖链若采用“析构 C → 释放 P → 递归析构 P”的实现，会把链长度映射到 C++ 调用栈深度。当前实现用一个局部 pending 链迭代处理：取一个零引用 Work，把边数组交换到局部 vector，销毁当前 Work，再逐个释放其前驱引用；新归零的前驱挂到 pending 上。

pending 借用已归零 Work 的 `m_parent` 字段，不额外分配回收节点。这在该对象已退出正常执行期、原父完成语义不再需要时成立，不能把运行中的父指针拿来当临时工作链。

先接管边表、再销毁 Work、最后释放前驱引用还有一个重要结果：前驱的寿命覆盖当前载荷的整个析构过程。用户 callable 的析构若合法依赖此前保留的数据，不会因为前驱引用提前释放而在中途消失。

图内普通 Work 则由 Graph 生命周期管理，不以同样的异步引用链独立销毁。队列不拥有 Work；引用和图所有权必须在发布之前已经建立，不能指望队列帮忙延寿。

源码：`work.hpp:625-637,1688-1720`；`async_future.hpp:406-414`。

37 Invoker: 统一框架，不同执行阶段



这是普通图 Basic 的骨架；不同 Invoker 的钩子和停止检查不同。

图 37：普通任务的执行骨架

Basic 图任务的路径大致为：检查停止 → 尝试获取信号量 → 标记执行 → before 观察者 → 调用用户函数并捕获异常 → after 观察者 → 释放信号量 → 清执行标记 → 推进依赖并收尾。信号量获取失败会提前返回，Work 留在等待链中，既未执行用户函数，也未归还父 slot。

Payload 把任务类型映射到具体 Invoker。Branch/Jump 在用户函数中收集选择目标，随后进入对应 teardown；Runtime/SubFlow/Module 还需要首次进入和完成恢复两个阶段。统一 Work 不等于所有任务种类执行完全相同的钩子。

尤其要区分 Executor 的轻量 AsyncBasicInvoker 与可配置 AsyncTaskBasicInvoker：后者包含观察者和信号量框架，前者的快路径直接写结果、捕获异常并完成。不能以某个类的行为推断所有提交 API 都有同样的回调开销。

TFL_ENABLE_WORK_EXECUTION_CHECK 启用时，Work 控制位记录是否正在执行；重复进入或不匹配退出触发 terminate。这是诊断性断言机制，不是用来把错误的并发激活变成合法串行执行的锁。

before/after 的统计边界还会受任务类型影响：对等待子任务完成后才恢复的 Runtime 任务，它们覆盖整段逻辑任务生存区间，不能直接当成用户函数独占 CPU 时间。

源码：work_invokers.hpp:36-79,498-522,1301-1312,1555-1573。

38 Runtime：用哨兵分离函数返回与任务完成



若去哨兵时旧值已经是 1，当前线程直接收尾，不必重新调度。

图 38：Runtime 首次进入与恢复收尾

首次运行 Runtime Work 时，设置 PREEMPTED，给自己的 join 加一份哨兵，再调用用户函数。每提交一个子任务，先给父 join 增加一份，后发布子任务。子任务结束归还各自的 slot，但哨兵保证用户函数仍在运行时父计数不能变为零。

用户函数返回时释放哨兵。如果旧值为 1，子任务已经全部完成，当前线程可以直接收尾；否则返回调度循环，让其他工作继续执行。最后一个子任务把父 join 从 1 减到 0，发现 PREEMPTED，重新安排父 Work。

恢复后的 Invoker 看见 PREEMPTED，跳过用户函数，直接做 after、资源释放和最终 teardown。用户函数不会因此再调用一次，局部 C++ 调用栈也没有被保存成可恢复协程帧。

因此这里的“挂起/恢复”是库的逻辑阶段，不是操作系统抢占，不是 C++ coroutine 的 co_await。真正的用户函数必须先返回，或者在内部显式使用协作等待；一个长时间阻塞的函数仍占着 Worker。

图内 Runtime 的普通入度计数在取得激活权后已归零，同一个 join 字段随后被用作子任务计数。最终完成再恢复它的静态 join weight。理解字段的阶段复用，才能看懂为何不存在两份独立计数成员。

源码：work_invokers.hpp:794-833,1342-1368; runtime.hpp:331-402; executor.hpp:1745。

39 SubFlow 与 Module：动态图和复用图

SubFlow 的 Invoker 内部拥有 Graph；构造当前 SubFlow 上下文时清空旧图，用户函数在这一轮构建节点。SubFlow::run() 显式设置父关联、增加源节点 slot 并发布。仅创建节点并不会隐式启动它们，函数返回也不等同于替用户调用 run。



两者都需独占正在运行的物理 Graph 状态。

图 39：SubFlow 构建/启动与 Module 循环

SubFlow 采用与 Runtime 相同的 +1 哨兵：用户构图和提交期间防止父提前完成；run 后可以显式 wait，也可以返回，让剩余子节点完成后恢复父的收尾阶段。再次运行期间不能重复调用 run 来并发执行同一批节点。

Module 则绑定一个图持有者，可能拥有图，也可能借用图。首次进入只初始化图和源点个数；每次恢复检查继续条件，若继续则建立源节点计数，把一项放入 cache、其余排入队列。无需在每轮重新构建所有 Work。

两者都复用父 Work 的阶段状态，却有不同的存储寿命：SubFlow 的动态图由其载荷拥有，下一次首次进入会清理；借用模块图必须由调用者保持到运行结束。一个 Future 只延长异步 Work 的寿命，不能自动延长任意外部借用图对象。

设计取舍：动态图方便按运行时数据构建依赖；复用图减少分配与建边成本。它们都需要独占可变图状态，不能用一个物理 Graph 承载两次重叠运行。

源码：subflow.hpp:62-84；work_invokers.hpp:868-907,974-1025,1470-1510。

40 协作等待：等孩子时继续帮执行器工作



协作等待仍不能解决“持有子任务需要的锁再等待它”的资源死锁。

图 40：阻塞等待与协作等待的单 Worker 差异

假设只有一个 Worker，父任务启动一个子任务后在普通 `Future::get` 上阻塞。子任务虽已排队，却没有空闲 Worker 执行它，这会形成线程池饥饿式等待。`Runtime::wait/corun/wait_until` 使用 `_corun_until`，等待期间仍从本地队列取任务、窃取其他目标并执行。

协作循环先检查目标谓词，再 pop 自己的工作；没有工作则探索 victim，并在预算后 yield。它不进入 `Notifier` 的普通休眠协议，而是持续确认等待条件，避免为一个仅靠当前协作执行推进的目标永久停泊。

`Runtime/SubFlow::wait` 等 `join==1`，因为调用者仍处于用户函数内部，自己的哨兵尚未释放。`TaskGroup` 的独立锚点没有这份执行哨兵，所以等 `join==0`。把二者阈值混用会造成提前返回或永不结束。

`corun` 使用局部 `AnchorWork` 聚合这一段子图的完成和异常。这个锚点在栈上，必须在子工作结束前保持有效；协作等待就是覆盖其寿命边界的机制。

协作执行也有代价：等待函数期间可能执行其他任务，应用必须能接受这种重入式调度；嵌套 `corun` 会增加普通调用栈深度。它不解决用户锁顺序错误：持锁等待一个也需要该锁的子任务，协作执行仍会死锁。

源码：`executor.hpp:1778-1830`；`runtime.hpp:581-605`；`subflow.hpp:79-84`。

41 TaskGroup: 把一段子任务收拢到局部锚点

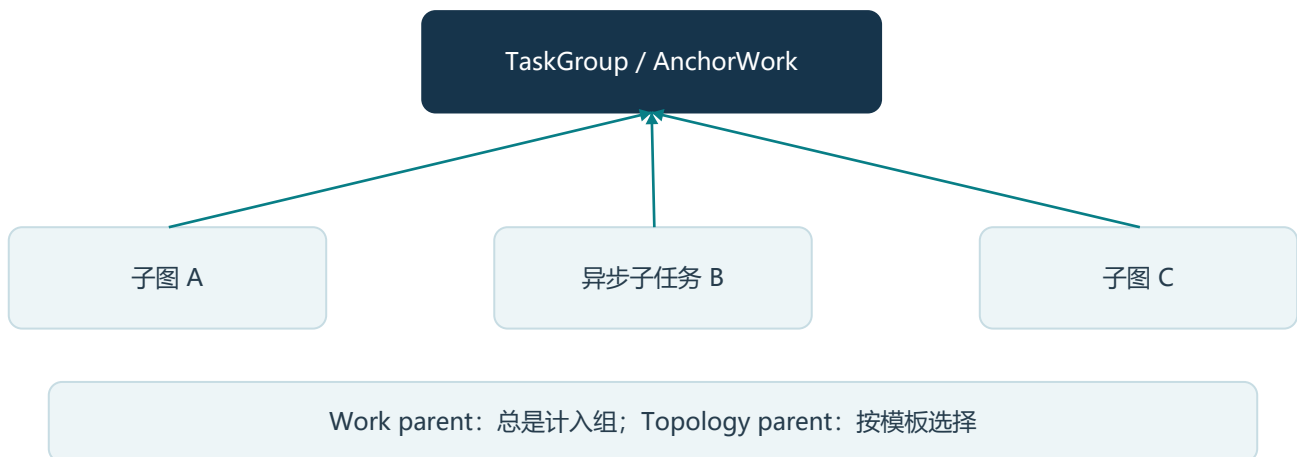


图 41: 局部完成锚点与可选的停止继承

TaskGroup 绑定当前 Context 的 Worker 和 Executor，并在组对象内放置一个 AnchorWork。组内提交的任务把这个 anchor 当作父 Work；提交前增加它的 join，完成后归还。组的 wait() 协作执行到计数零，再重新抛出记录在本组锚点的异常。

析构函数是 noexcept，只协作等待计数零，不调用重抛。这样栈展开期间不会因为第二次异常导致意外 terminate；代价是如果业务需要观察组级异常，就应在合适的捕获边界显式 wait。

InheritTopology=false 是当前模板默认值。它控制子任务是否以组锚点 Topology 为停止继承父链，不影响 Work 完成父链。特别要避免照抄旧注释，把“关闭继承”解释成子任务脱离本组生命周期。

异常传播则需按任务种类阅读：异步 Work 自己是显式异常锚点，异常可由返回 Future 观察；直接在组锚点下执行的图节点会沿 Work 父链汇聚到组。不能宣称所有子 Future 的异常都会自动在 TaskGroup::wait 再抛一遍。

启用停止继承还会引入非拥有 Topology 父指针的寿命约束。即使子任务已完成，保留的句柄若仍查询继承停止状态，相关祖先 Topology 也必须有效。引用计数只保护对应异步 Work，不自动把整个祖先链都变成共享所有权。

源码：task_group.hpp:305-392, 579-600; work.hpp:1613-1642, 796-861。

42 多信号量获取：全检查后统一扣减



若所有资源足够，才进入第二遍统一扣减。

图 42：按全局顺序锁定多个资源并事务式获取

Work 的 `acquire` 请求按 `std::less` 的指针全局顺序排列并合并同一信号量请求。尝试获取时，`SemaphoreLock` 按这个顺序锁住全部资源；第一遍只检查每个剩余配额是否足够，不做扣减。只有全部满足，第二遍才统一减掉所需数量。

例如任务需要 $A \times 2$ 、 $B \times 1$ ，而当前 $A=3$ 、 $B=0$ 。检查会在 B 失败，此时 A 仍为 3，不存在“先占 A、发现 B 不足、再回滚 A”的中间状态。当前实现已采用这种两阶段协议，不能把历史版本逐个获取回滚的分析作为本版事实。

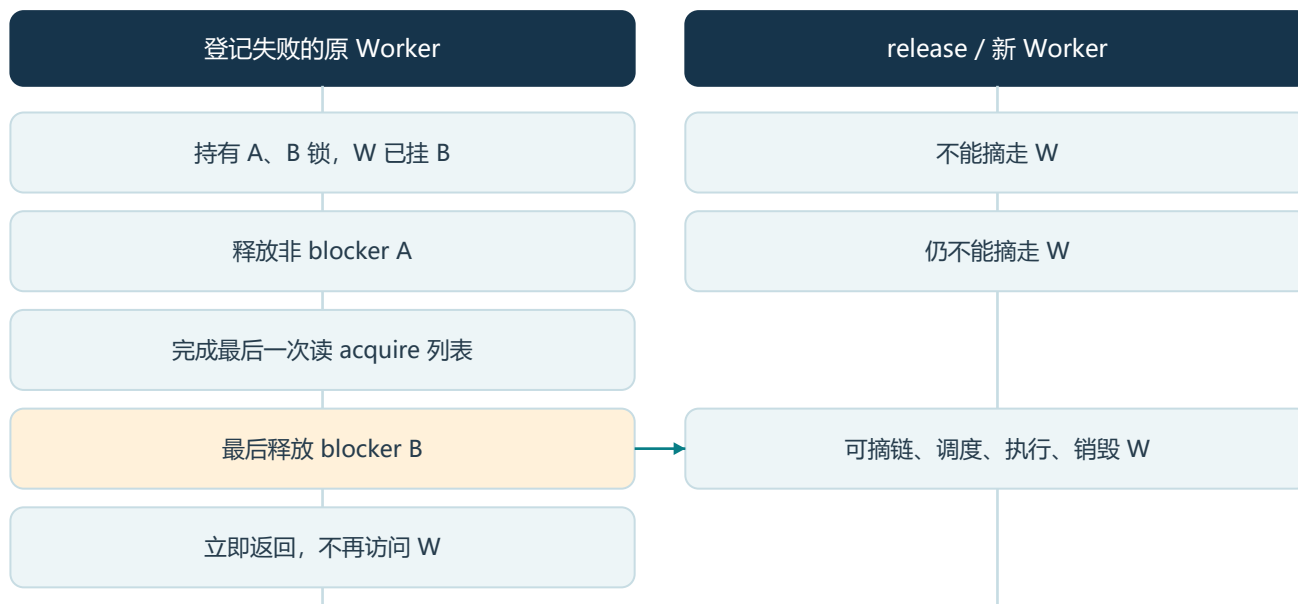
固定锁顺序排除了内部多资源获取的循环锁等待：所有任务都沿同一个资源排序取得锁。它不保证业务层无死锁，特别是任务持有配额后等待另一个需要同样配额的子任务，依然可能形成资源循环。

失败任务只挂到第一个不足资源 blocker 的等待链。它没有占用任何资源配额，但仍持有父 slot 和执行期寿命；稍后被重新发布时必须从头获取整个资源集合，不能以为唤醒就等于已获授权。

设计取舍：一次锁住 k 个资源让获取具备整体提交语义，也扩大短临界区和热点耦合。适合数量较少、临界区短的资源约束；不是高竞争大资源集合的免费方案。

源码：`work.hpp:198-212,1399-1418,1483-1536`。

43 为什么必须最后解锁 blocker



B 解锁是允许其他线程重新取得 Work 的寿命边界。

图 43：最后一把锁也是 Work 的寿命边界

获取失败后，当前 Work 已挂入 blocker 的 waiter 链。只要 blocker 的锁还持有，其他线程就不能把它摘出重新调度。问题在于 SemaphoreLock 析构仍要遍历 Work 内部的 acquire 列表，释放其余资源锁。

若先解锁 blocker，另一个线程可以 release 资源，摘出 Work 并发布；它可能立即在另一 Worker 上完成，甚至被销毁。原线程却还在遍历同一个 Work 的 acquire 数组，这会变成寿命竞争，而不仅仅是“配额会不会算错”。

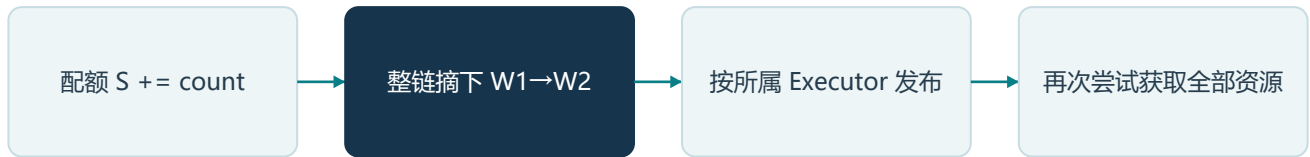
当前析构因此先把 blocker 保存到局部变量，逆序释放所有其他锁；结束对 acquire span 的最后一次访问后，才解锁 blocker，并立即返回。这个最后解锁操作就是允许 Work 离开当前等待登记阶段的边界。

这里的安全性无法通过把 acquire 元素改成 atomic 来替代。对象已经析构时，原子的内存位置同样不再可访问。必须用顺序和所有权防止过早重新发布。

类似模式也出现在其他位置：异步完成先缓存 parent 再释放最后引用；Notifier 先缓存 next 再唤醒可能重用 Waiter 的线程；共享栈先清 next 再把 Work 交出去。它们共同遵守“跨越发布或回收边界后，不再读旧所有者依赖的数据”。

源码：work.hpp:222-240,1515-1536; notifier.hpp:599-609。

44 release: 整链唤醒，重新竞争资源



没有逐个预留配额；失败者可能换一个 blocker 再等待。

图 44：归还配额时摘下整条等待链

`Semaphore::_release` 在内部锁下先增加可用配额，再把整条 waiter 链 $O(1)$ 追加到输出链，清空内部头尾。它没有逐个检查每个等待者这次能否成功，更不会预先把配额分配给队首。

随后 `_schedule_from_semaphore` 遍历输出链，先保存 next，再清当前节点 next，按 Work 自己的 Topology 找到所属 Executor。同池工作进入当前 Worker 的本地调度；不同池工作进入它自己的外部共享入口。

被唤醒的多个任务再次执行完整的多信号量获取协议。假设释放 1 份资源、等待者有 10 个，它们都可能重新被发布，但最多满足配额约束的任务通过；其余重新挂到新的 blocker。这是一种整体重试策略，不是每次 release 精确唤醒一位满足全部资源的任务。

为什么这样做？避免在单个信号量锁下尝试获取等待者的所有其他锁，从而形成复杂锁顺序和跨资源判断。代价是热点资源上的重复唤醒和争用，可能出现惊群及饥饿；waiter 链入队顺序也不能升级为严格公平的获配顺序。

配额上界以断言检查，release 不应超过最大值；acquire/release 配置要符合业务协议。取消一个任务并不会自动为它创建业务资源的补偿事务。

源码：work.hpp:1539-1547, 1566-1611；executor.hpp:1757-1774。

45 SpinMutex: 短锁也需要减少争用写

算法摘写 / 保留关键操作顺序

```
if !flag.test_and_set(acquire): return
spin = 1
loop:
    while flag.test(relaxed):
        repeat spin times: CPU_RELAX()
        if spin < 64: spin *= 2
        else: this_thread.yield()
    if !flag.test_and_set(acquire): return
unlock: flag.clear(release)
```

无竞争时只需一次 `test_and_set`。竞争时，先做只读 `test` 轮询，看到可能空闲再尝试有写入性质的 RMW。若每个失败者都不断 `test_and_set`，会让同一缓存行在多个核心间持续争夺写权限，放大本来很短的临界区。

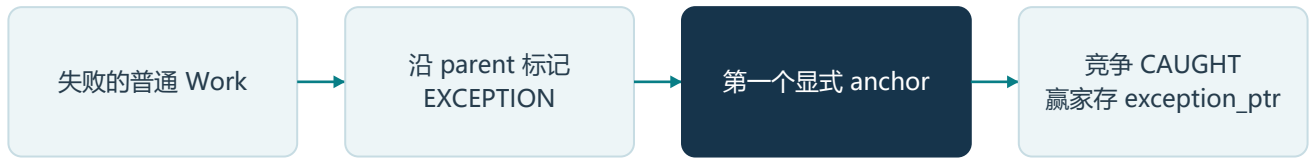
退让长度按 1、2、4、8、16、32、64 增长，到上限后 `yield`。`CPU_RELAX` 是架构相关的自旋提示，`yield` 则请求线程调度器给其他线程机会，两者都不保证某个等待者下一次必然获得锁。

`release` 清锁与后继成功 `acquire` 形成临界区的数据传递。仅在轮询中使用 `acquire` 不能代替最终成功取得锁；也不能用 `relaxed clear` 发布临界区内普通写入。

这种锁适合信号量计数和等待链等短操作。若锁内做长时间阻塞、I/O 或用户回调，竞争线程会浪费 CPU，持锁线程被抢占时延迟也会放大。当前锁不递归、不保证公平，不具有 lock-free 进展性质。

源码: `spin_mutex.hpp`:63-104; `work.hpp`:198-212,1569。

46 异常如何寻找归档位置



没有显式锚点才用最近隐式锚点；可见性依靠随后的完成同步。

图 46：沿 Work 父链寻找异常锚点

用户 callable 抛异常后，Invoker 捕获 exception_ptr，沿 Work 的 m_parent 链向上搜索。遇到第一个显式锚点就停止；沿路普通节点置 EXCEPTION，并记录最近的隐式锚点。显式锚点优先，只有没有显式锚点时才选择已找到的隐式锚点。

归档用 `fetch_or(EXCEPTION|EXCEPTION_CAUGHT, relaxed)` 竞争一次写入权。第一个发现 CAUGHT 原先未置位的线程保存 exception_ptr；其他异常不会并发覆盖同一异常指针，而会走各自节点的备用归档路径。它并不是一个保存所有异常的集中列表。

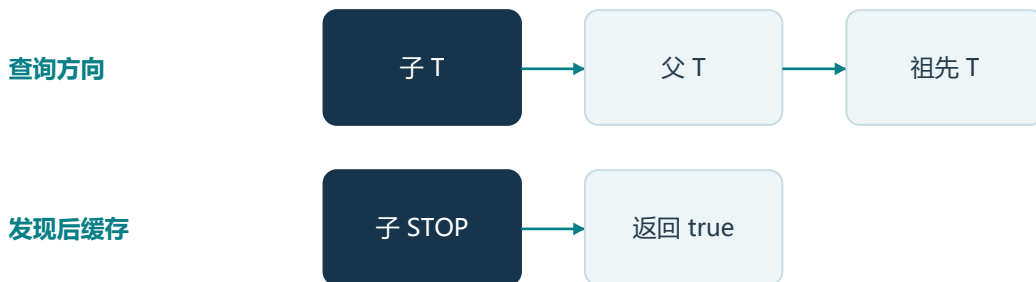
relaxed 位操作只是仲裁归档者，并不单独发布 exception_ptr。观察异常之前还要经过完成计数的 acquire 或 Finished 的 acquire 等待，让指针写入可见。若只轮询 EXCEPTION 位后立刻读取指针，就绕过了真正的完成同步边界。

普通图节点出现异常时，完成路径不再向后传播，并归还当前父 slot；其他已经活动的分支仍需按协议结束。异步 Work 自身是显式锚点，其 Future::get 等完成后读取共享异常；异步依赖只表示完成顺序，不自动把前驱异常复制到后继。

设计边界：异常传播沿 Work 父链；停止继承沿 Topology 父链。这两条链不能混讲，尤其不能把 `InheritTopology=false` 当成统一切断所有完成和异常关系的开关。

源码：work.hpp:796-861；executor.hpp:1311-1315；async_future.hpp:347-363。

47 停止请求是协作标记，不是线程终止



Topology 父指针是非拥有关系；停止不等于抢占线程或回收任务。

图 47：停止查询沿 Topology 祖先链向上查找

`request_stop()` 用 `relaxed fetch_or` 设置当前 Topology 的 `STOP_REQUESTED`，返回本次是否第一次置位；它不遍历所有后代，也不调用操作系统去杀死 Worker。`stop_requested()` 从当前 Topology 沿父链查询，发现祖先停止后把标记惰性缓存到当前拓扑。

这把广播成本从请求时转移到查询时：请求只改一个字，后代按需要向上观察。父链是非拥有指针，必须在查询期间保持有效；缓存能缩短后续查询，却不能成为释放祖先的通用寿命协议。

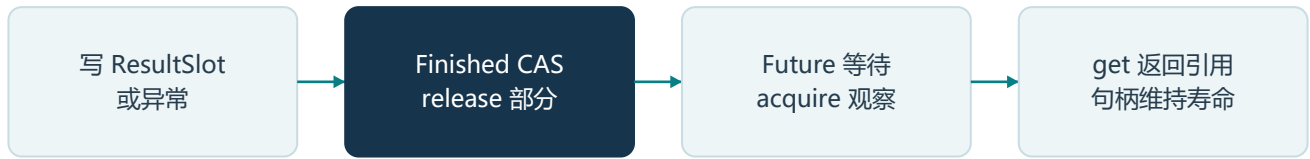
普通图 Invoker 在进入用户 callable 前检查停止，若已请求，跳过该段执行并归还父 slot。Runtime 用户函数可以在长循环中主动检查停止。不同异步 Invoker 的检查点并不完全相同，不能承诺每个已排队的基本异步 callable 都会因停止请求而自动跳过。

停止不会抢占正在运行的函数，不会立即摘出信号量等待链，也不替应用归还任意业务资源。若永远不会出现释放动作的资源等待者仍占着 slot，仅请求停止可能不足以让整个执行器排空。

Executor 的 `terminate` 位是 Worker 生命周期控制，在 `wait_for_all` 之后用于结束工作线程；它与用户 Topology 的 `STOP_REQUESTED` 不是同一条状态机。调度关闭和业务取消必须分别分析。

源码：`work.hpp:686-719`；`work_invokers.hpp:498-504,1301-1311`；`executor.hpp:1092-1110`。

48 ResultSlot 与 Future: 存储不负责同步



结果容器本身没有同步；引用结果不拥有被引用的外部对象。

图 48: 结果先写入, Finished 再发布, 最后按引用回收

ResultSlot 的普通值存储有两种策略。默认可默认构造且可由右值赋值的 R 使用直接成员, 执行后赋值; 不满足或关闭偏好时使用 optional, 在结果产生时构造。左值引用结果只保存对象地址; void 没有实际返回值存储。

ResultSlot 自己没有锁, 也没有“结果就绪”的原子位。它依赖外层 Topology 完成协议: Invoker 先存结果或异常, 完成路径 release 发布 Finished, Future::wait acquire 观察结束, 之后才能访问结果。

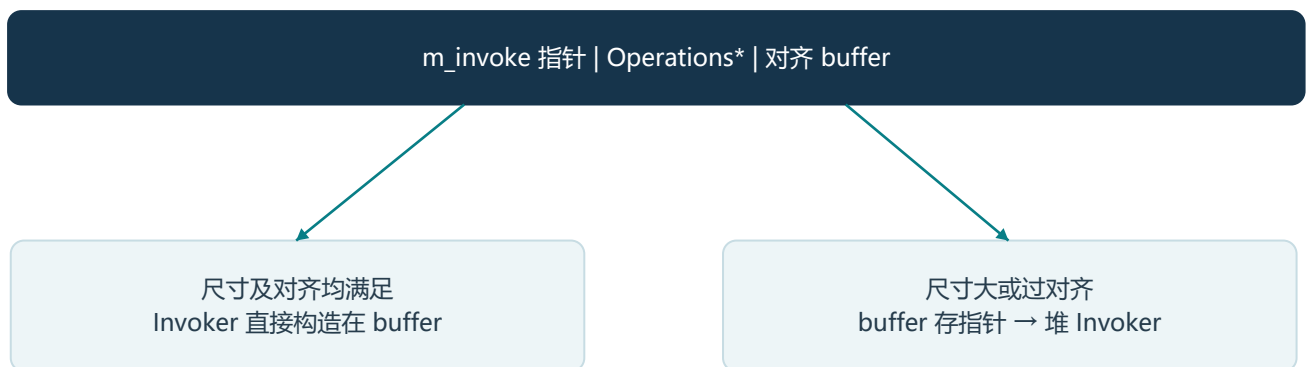
AsyncFuture::get() 是非消费式读取: 等待完成、检查共享异常, 然后通过 const ResultSlot 返回 ref。普通值通常得到 const R&, 同一个 Future 可重复 get; 引用类型则仍是所指对象的引用, void 只做完成与异常观察。不要套用 std::future 的“一次 get 后句柄失效”语义。

返回的值引用由底层 Work 的 ResultSlot 持有。最后一份强引用消失会回收 Work, 因此缓存 get 的引用后再销毁所有句柄, 引用会失效。对于 R=T&, Future 更不会拥有 T 本身, 它只保留一个地址。

多个独立句柄可以观察同一结果, 但同一个句柄对象不能一边 reset/移动/析构, 一边被另一线程读取。共享状态的同步不等于句柄对象自身可以任意并发修改。

源码: result_slot.hpp:27-36,44-116,129-222; async_future.hpp:340-355; work_storage.hpp:52-90。

49 Payload: 把调用快路径与冷操作分开



默认 128 是尺寸参数；64 位下 buffer 通常为 112 字节。

图 49：内联载荷与堆载荷的两种布局

Payload 由一个直接调用函数指针 `m_invoke`、一个 `Operations` 表指针和对齐缓冲区组成。`Operations` 保存 `destroy`、`dump` 与任务类型；热路径 `invoke` 不必先经过完整操作表再找调用入口。

默认 `TFL_WORK_PAYLOAD_SIZE=128`，内部 buffer 大小是该值减去两个指针大小。常见 64 位目标下为 112 字节；是否内联还要检查 `Invoker` 的对齐不超过 `max_align_t`。这里比较的是包含 `callable`、结果或图持有者等状态的整个 `Invoker`，不只是 `lambda` 捕获大小。

小对象 `placement construct` 到 buffer；大或过对齐对象独立 `new`，buffer 里用 `memcpy` 保存其指针。构造成功后才发布操作表和调用指针，避免析构入口指向未成功构造的对象。`Work` 本身不可移动，减少了类型擦除载荷重定位协议。

代价是每个 `Work` 都预留 buffer 空间，增大阈值可减少堆分配，但会扩大节点与对象池 `slab` 的工作集。调整不能只看“内联率更高”，还要看缓存容量、节点数量和实际 `Invoker` 分布。

当前异常边界。 `Payload::emplace` 先 `reset` 旧载荷，再构造新载荷；若新构造失败，旧任务不会自动保留。因此 `Task::work` 载荷替换没有强异常保证。需要保留旧 `callable` 的场景，应以新节点准备成功后再切换结构，而不能假定失败等同于未修改。

源码：`work.hpp:276-292,313-318,350-459,504-519`；`macros.hpp:198-202`。

50 对象池：块、slab 与分片

ObjectPool<Work, 32, 64>：线程内计数器轮转选桶



ObjectBlock = 原桶指针 + atomic next_free + 对齐的 Work 存储。

图 50：32 个桶，每个初始 slab 含 64 个块

默认 Work 池是 ObjectPool<Work, 32, 64, ...>。每个桶有一个空闲块栈、一把补充 slab 的 mutex，以及 slab 链。池构造时每桶预分配一个 slab，因此初始是 $32 \times 64 = 2048$ 个块，不是首次使用某个桶时才分配。

ObjectBlock 在 T 的原始存储之外保存 free_stack 归属指针和原子 next_free。create 从空闲栈取块，再 placement construct T；destroy 先析构 T，再按块记录的归属返回原桶。跨线程销毁不需要猜测最初由哪个线程分配。

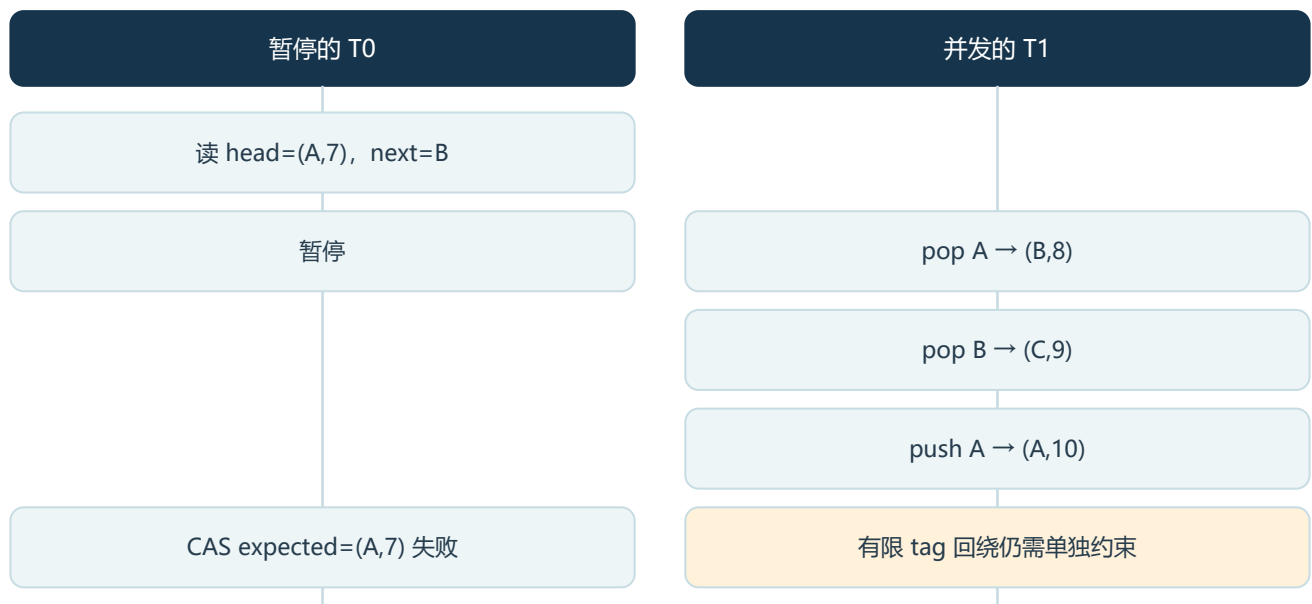
桶选择使用 thread_local 计数器，以线程 ID 哈希初始化，随后每次递增并按二次幂桶数取掩码。这是线程内轮转分散，不是一个固定的线程私有池。消费者和销毁线程都可能操作同一桶的空闲栈。

块元数据与 T 存储分离非常重要：T 已析构并重建时，落后的空闲栈 pop 仍可能读取 next_free。把链接埋进 T 的活动存储会把对象寿命问题与空闲链并发读混在一起。

slab 保留到桶析构才释放。这样快路径不会遭遇其他线程已经释放 slab 的悬空元数据，但高峰后内存也不会自动回落到当前活跃 Work 的规模。池销毁时必须确保没有并发访问和未归还的活动对象。

源码：object_pool.hpp:204-362, 383-423；work.hpp:1651-1659。

51 Tagged head: ABA 为什么需要版本



版本防止把“相同地址”误当“相同状态”；不是无限历史的绝对防 ABA。

图 51：指针回到 A，但版本从 7 变为 10

只用指针 CAS 的空闲栈会遇到 ABA：T0 读到 head=A、next=B 后暂停；T1 取走 A、B，再把 A 放回，head 又是 A。T0 若只比较地址，会误以为状态没变，可能把已经被取走的 B 接回空闲栈。

当前 head 打包指针与 tag，每次 push/pop 都推进 tag。T0 的 expected (A,7) 不再匹配 (A,10)，必须重读。next_free 是 atomic，解决落后线程读取链接与块再入栈修改链接之间的数据竞争；tag 则解决逻辑版本问题。

优先选择的 TaggedHead128 使用 64 位指针加 64 位 tag，前提是其 atomic 在目标上始终无锁；否则选 TaggedHead64，把地址压到低 PointerBits，剩余高位作 tag。默认 PointerBits=48 时，tag 只有 16 位。

边界不能省略。 有限 tag 会回绕；若一个滞后的操作跨越同一桶足够多次修改，旧 (pointer,tag) 仍可能再次出现。因此“带 tag”不是无限执行期 ABA 的绝对证明。128 位版本也有有限回绕，只是空间大得多；完整证明需要滞后上界或更强回收/版本协议。

64 位压缩版要求指针可编码，代码以断言验证。它只接受不超过 48 的 PointerBits；不能照宏旁旧注释直接设为 52/57，就宣称当前池支持相应地址空间。

源码：object_pool.hpp:35-176,487-534；macros.hpp:231-262。

52 对象池快路径与慢路径的成本

算法摘写 / 保留关键操作顺序

```
create:
    bucket = next_thread_local_bucket()
    block = pop_tagged_free_stack(bucket)
    if no block:
        lock(bucket.refill_mutex)
        retry pop
        if still empty: allocate slab, publish remaining blocks
    try construct T in block
    on exception: return block to its free stack; rethrow
```

pop 先 acquire 读取 tagged head, 读取 next_free, 再 CAS 成下一个指针与新 tag; push 先写尾节点 next_free, 再 release CAS 发布链。slab 初始化完成后, 空闲块链也通过相同发布方式交给其他线程。

慢路径加桶内 mutex 后再次尝试 pop, 避免等待锁期间别人已经归还了块, 却仍然多分配 slab。确实缺块时才 new slab, 取第一块供当前请求使用, 其余整链一次压入空闲栈。

构造 T 抛异常时, 块会被放回, 防止“分配成功、对象构造失败”泄漏池容量。这份局部异常保证不代表上层 AsyncTask 动态链接也有同样回滚能力; 需要分层分别审查。

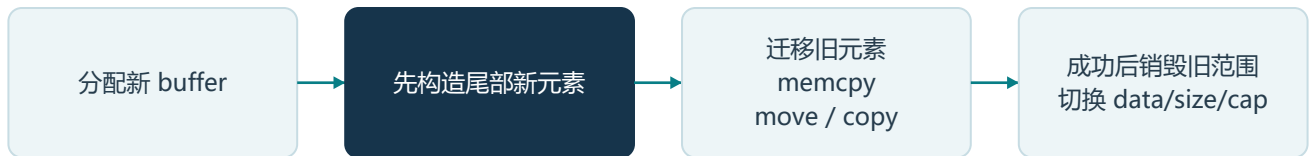
不能把整个对象分配过程称为 lock-free。即使 free_stack 原子类型本身无锁, 补充 slab 有 mutex, 底层内存分配也可能阻塞。free_stack_is_always_lock_free 只报告相关原子原语条件, 不包含 tag 回绕的算法正确性或慢路径进展。

启用池降低反复构造大量小任务的系统分配频率; 关闭池便于隔离池相关问题和比较峰值保留内存。性能结论应分别统计热桶争用、补 slab 次数、对象构造成本及高峰后的保留量。

源码: object_pool.hpp:346-348,383-469,487-547; work.hpp:1670-1685。

53 SmallVector 与随机数：辅助结构也有算法

SmallVector 默认内联容量 $N=4$ ，保存 data、size、capacity 和内联存储。小目标集合可免去独立分配，超过容量才迁移到堆。普通对齐用 malloc/free，过对齐类型用 aligned new/delete；这些分配方式必须配对，不能统一 free。



先构造新增元素可保护引用自身元素的参数；失败保证依赖 T 的能力。

图 53：先构造新增元素，再迁移旧范围

扩容 `emplace_back` 先在新缓冲区尾部构造新增元素，再迁移旧范围。这能处理参数引用自身元素的情况：若先搬走或销毁旧数据，新增元素的构造参数可能已经失效。可平凡迁移类型用 `memcpy`；其他类型逐个 `move_if_noexcept`，并清理失败时已经构造的目标元素。

这种处理也有边界：只有可能抛出的移动、没有可用复制的类型，迁移失败后原元素可能已部分改变；不能把所有泛型操作一律标成强异常保证。SmallVector 也不是线程安全容器；多目标收集依靠当前执行阶段的独占访问。

SplitMix64 为每个 Worker 保存私有 64 位状态，以固定增量和移位/乘法混合生成输出。范围映射使用 `mulhi64(random,n)`，避免每次除法取模；若输入均匀，各桶对应输入数量最多差一，不必宣称数学上完全无偏。

`mulhi64` 在支持的平台使用 128 位整数或编译器 intrinsic，兜底用四次 32×32 乘法与进位合成高半部。它还用于共享分片缩放；该随机源不用于安全令牌。

源码：small_vector.hpp:113-134,1135-1203,1320-1346；random.hpp:27-165。

54 缓存行、配置与平台前提

项目	当前默认/实现	为什么影响架构
本地容量	TFL_DEFAULT_QUEUE_SIZE=1024	影响每 Worker 指针数组与溢出频率
Payload 尺寸参数	TFL_WORK_PAYLOAD_SIZE=128	决定内联容量和节点工作集
Work 对象池	默认启用，32 桶×64 块/slab	降低分配频率，保留峰值 slab
缓存行尺寸	按平台宏选择，可覆盖	top/bottom、Waiter、桶的隔离
压缩地址位	常见默认 TF_POINTER_BITS=48	影响指针可表示范围与 tag 位数
执行检查	随配置/调试设置启用	诊断重复执行，增加控制字操作

对齐旨在减少伪共享：两个互不相关、却落在同一缓存行的变量被不同核心写入，会引发不必要的缓存一致性流量。把 top 与 bottom 分开有助于 Owner 与 thief 各自写边界；共享栈 incoming 与 state 分开有助于生产和消费分离。

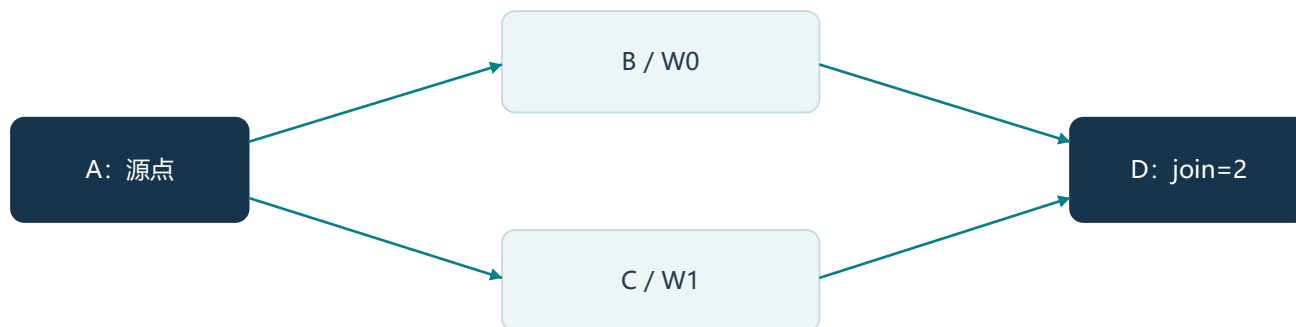
对齐也会增大对象与数组占用。更大的缓存行参数未必更好，特别是每 Worker 的 Waiter、池桶和局部队列数量较多时。应测量实际热点，而不是把所有成员一律各占一行。

头文件库的这些宏会改变类型布局或内联实现，必须在所有翻译单元一致配置，避免 ODR 和 ABI 不一致。当前 ObjectPool 中的 tagged head 明确要求 64 位指针；不能仅因 CMake 写 C++20 就推断所有 32 位目标也能编译运行。

本书不给虚构的纳秒级性能数字。复杂度只描述操作规模；实际收益受 CPU、内存布局、编译器、任务粒度、系统负载和资源竞争影响。

源码：macros.hpp:21-67,170-262；bounded_queue.hpp:122-136；object_pool.hpp:73-76,146-150。

55 全过程演算：菱形图的每一份责任



P 的 slot 轨迹: $1 \rightarrow 2 \rightarrow 1 \rightarrow 0$; 其中一个后继继承原责任。

图 55: P 包装 $A \rightarrow \{B, C\} \rightarrow D$, 一次合法执行轨迹

设 P 是单次运行图的模块 Work; A 是唯一源点, B、C 各有一条 strong 前驱, D 有两条。初始化得到 $B.join=C.join=1$ 、 $D.join=2$; P 为 A 建立一份 slot。这里展示一种合法交错, 不声明队列一定产生这个顺序。

时刻	执行动作	P 的活动 slot / 关键计数
1	P 把 A 放入 cache	$P=1$
2	A 使 B、C 都就绪	$B/C.join=0$; P 先增为 2
3	B 继承 A 的 slot, C 入队	B 在 W_0 , C 可由 W_1 窃取
4	B 完成, D 尚未就绪	$D:2 \rightarrow 1$; B 归还 slot, $P:2 \rightarrow 1$
5	C 完成, 取得 D 激活权	$D:1 \rightarrow 0$; D 继承 C 的 slot, $P=1$
6	D 完成且没有后继	恢复 $D.join=2$; $P:1 \rightarrow 0$
7	最后归还者恢复 P	predicate 结束; 发布 P 的 Finished

W_1 可以在 C 上写结果, B 也可以写自己的结果; D 的最后到达 RMW 链把两侧结果汇合。如果 C 先完成, 则对称地由 B 成为 D 的激活者, 父计数轨迹仍满足守恒规则。

P 发布 Finished 后, Future 可读取本次结果; 执行器稍后完成剩余收尾并减少顶层活动计数。若随后所有 Future 引用也释放, P 的 Work 才能回收。图中 A-D 的存储则遵守图持有者的寿命。

源码: `work_invokers.hpp:1470-1510`; `executor.hpp:1286-1365, 1617-1678, 1745-1755`。

56 全过程演算：异步依赖加资源等待



等待资源期间仍是 Running；执行引用和活跃计数没有提前归还。

图 56：前驱完成后就绪，资源不足时暂停，再唤醒执行

设公开 AsyncTask C 依赖已启动前驱 P，并需要信号量 S 的一份配额，初始 S=0。C.start 建立前驱引用、执行引用和活跃计数，令 C.join=2，其中一份来自 P，一份是登记哨兵。

如果 P 在链接前已 Finished，登记方直接把 C.join 从 2 减到 1；若 P 尚 Running，则将 C 登记到 P 的后继前缀，稍后由 P 完成路径减一。两条路径互斥。提交方最后释放哨兵，唯一把 join 减到零的一方发布 C。

Worker 取到 C 后，进入 AsyncTask 的 Invoker，尝试获取 S。因配额不足，C 挂到 S 的 waiter 链，函数直接返回；C 尚未执行 callable，状态仍为 Running，执行引用和顶层活动计数仍然存在。

另一条任务的 release 将 S 增到 1，摘下整条 waiter 链并重新发布 C。新的 Worker 再次完整获取资源，成功扣为零，调用 C 的函数、存结果，并执行 C 配置的 release 动作。资源唤醒提供的是重试机会，不是提前分配的令牌。

C 发布 Finished、通知 Future、推进自己的后继，再释放执行引用并减少活跃计数。用户释放最后句柄后回收 C，进而释放它保留的 P 引用；P 若因此归零也进入迭代回收链。

若释放 S 的任务永不运行，整个过程就停在资源等待阶段。停止标记不能凭空补出这次业务释放，必须从资源依赖图排查。

源码：async_task.hpp:338-446；work_invokers.hpp:1555-1573；work.hpp:1483-1611。

57 成本模型：优化省掉了什么，又增加什么

路径	主要工作量	取舍与容易忽略的成本
本地 push/pop	常数级槽/边界操作	最后一项 CAS; SC fence; 溢出回调
单次 steal	常数级读取加一次 CAS	失败重试在外层, 缓存跨核流量
共享 push	CAS 重试, 批次先构链 $O(n)$	分片热点, 消费者 gate 阻塞进展
图完成传播	遍历出度 $O(\text{outdegree})$	join RMW、就绪分支发布
图初始化	遍历节点与前驱 $O(V+E)$	源点重排、计数重置
动态依赖	$O(\text{依赖数}) + \text{向量扩容}$	控制字争用、引用保留
多资源获取	$O(k)$ 次加锁、检查、扣减	热点耦合、整链重试惊群
池分配	常数级快路径, 慢路径补 slab	mutex、系统分配、峰值内存保留

对于微小任务, 调度成本可能超过有效计算。cache 接力、内联载荷、批量发布和对象池都在降低这部分固定成本, 但它们解决的不是同一个瓶颈: 接力减少队列跳转, 内联减少载荷分配, 池减少 Work 分配, 分片减少共享入口争用。

对于不均匀的重任务, 窃取有助于改善负载分布; 对于阻塞 I/O, Worker 被占住的时间可能主导表现, 增加无锁结构并不能消除这一点。任务图关键路径也给并行度设限, 更多线程不能让严格串行依赖同时执行。

经典工作窃取的理论界限依赖特定任务模型、调度假设和无阻塞条件。当前库还包含 Jump、资源等待、共享 gate 和用户阻塞, 不能未经证明直接套用某个最优期望时间公式作为本库保证。

源码: 对应第 05-54 节各条实际路径; 本节为基于实现的成本分析, 不是性能基准报告。

58 已知边界与架构审查重点

分配失败事务尚不完整。 `AsyncTask::start` 在取得 `LOCKED` 后调用前驱 `vector::reserve`；该区域没有覆盖全部失败路径的解锁回滚。动态登记 `_link_predecessors` 在前驱 `LOCKED` 内 `push_back`，扩容异常可能留下锁或已建立的部分状态。它们不是普通用户函数异常的处理路径，不应被“所有异常都由 `Future` 接住”掩盖。

载荷替换不是强异常保证。 `Payload::emplace` 先销毁旧对象。失败后保留旧任务需要额外的准备后提交设计，当前实现没有承诺。

序号协议存在前提。 有界队列逻辑索引不得溢出；`Notifier epoch` 的先后判断依赖半范围假设；`TaggedHead64` 的有限 `tag` 可能回绕。后两项属于需要进一步模型检查和约束论证的范围，本文未把它们写成已通过无限交错证明。

生存期不全由智能句柄托管。 图借用、`Topology` 父指针、`ResultSlot<T>` 和观察者回调捕获都有外部寿命条件。一个 `Future` 持有其 `Work`，并不拥有所有被该 `Work` 引用的外部对象。

调度没有实时/公平保证。 长接力链、共享消费者 `gate`、资源重试和用户阻塞都影响个别任务进展。图内 `Jump` 的合法静态形状也不代表安全重入或必然终止。

这些结论分别来自源码中可定位的异常出口、有限字段和所有权协议。本文保留工作区实现，只记录限制；后续修复应分别增加失败注入、序号模型和生命周期测试，而不是用一个大范围“并发优化”补丁混合处理。

源码：`async_task.hpp:389-431`；`executor.hpp:1601-1610`；`work.hpp:315-318`；`object_pool.hpp:93-143`；`notifier.hpp:310-326`。

59 如何验证这些解释，而不制造虚假信心

本书采用三层验证。第一层是源码核对：只阅读实际启用的实现，逐一核实关键内存序、状态位、分支条件和生命周期出口，并记录核心文件指纹。图中的箭头、计数和线程归属与这些路径对应。

第二层是具体轨迹与针对性运行检查：菱形图验证 fan-out/fan-in 及重复运行；异步链验证登记哨兵与早完成路径；有界队列验证单 Owner、多 thief 的每项唯一取得；共享栈验证多生产者/多消费者传递；资源任务验证获取、等待、释放和排空。测试必须设置超时，失败时保留可定位输出。

第三层是文档交付检查：确认源稿章节完整、图号及图文件一致、PDF 目录可跳转、中文字体嵌入、所有页面实际渲染，检查文字越界、图内标签和表格分页。文字可提取不等于图形没有重叠，因此还需要逐页视觉检查。

不能据此宣称的结论：本次检查不是 ARM 等弱内存平台上的穷尽证明，不覆盖任意暂停时长、tag/epoch 全空间回绕，也不替代分配失败注入或完整测试套件。当前工作区多份原测试文件处于删除状态，不把旧套件通过记录当作本快照的验证结果。

从维护角度，建议把验证按不变量组织：唯一取得、发布可见、slot 守恒、单次启动、单次归档、无提前回收、prepare 必闭合。这样实现重构后仍可检验协议，而不会只是在测试中重复函数内部写法。

本次结果 (Windows x64, MSVC C++20, 优化构建且保留断言)：六组针对性检查全部通过。有界队列传递 250000 项，使用 1 个 Owner 和 4 个 thief，并检查批量前缀/溢出；共享栈传递 96000 个 Work，使用 4 个生产者和 4 个消费者，混合单项与批量发布；两者都验证每项恰好被取得一次。

菱形图复用运行 2000 轮；异步依赖登记检查 2000 次，交替覆盖已完成前驱和可能并发完成前驱；多信号量检查 2048 次激活，反向配置请求顺序并验证容量与排空；另完成 300 轮空闲后提交/完成循环。以上为有限运行证据，不宣称穷尽竞态或性能排名。

60 源码阅读路线与术语索引

需要回答的问题	首要文件与入口
Worker 如何取得下一项工作	executor.hpp: <code>_invoke</code> 、 <code>_wait_for_work</code> 、 <code>_corun_until</code>
最后一项由谁拿走	bounded_queue.hpp: <code>pop</code> 、 <code>steal</code>
外部任务如何发布	shared_work_stack.hpp: <code>push</code> 、 <code>steal</code> ; executor.hpp: <code>_push_shared</code>
为什么不丢唤醒	notifier.hpp: <code>prepare_wait</code> 、 <code>commit_wait</code> 、 <code>cancel_wait</code> 、 <code>notify_one</code>
普通后继何时就绪	executor.hpp: <code>_set_up_graph</code> 、 <code>_tear_down_task</code>
分支与跳转差别	executor.hpp: <code>_tear_down_branch_task</code> 、 <code>_tear_down_jump_task</code>
动态依赖与完成竞态	async_task.hpp: <code>start</code> ; executor.hpp: <code>_link_predecessors</code>
子任务如何恢复父任务	work_invokers.hpp: <code>RuntimeInvoker</code> ; executor.hpp: <code>_schedule_parent</code>
资源等待的所有权边界	work.hpp: <code>SemaphoreLock</code> 、 <code>_try_acquire_semaphores</code>
结果何时安全读取	result_slot.hpp; async_future.hpp: <code>get</code> ; work.hpp: <code>_wait</code>
引用与内存如何回收	work.hpp: <code>_destroy_async</code> ; object_pool.hpp: <code>create</code> 、 <code>destroy</code>
异常及停止分别沿什么链	work.hpp: <code>_process_exception</code> 、 <code>_stop_requested</code>

Owner: 本地队列唯一 `push/pop` 线程。**thief/victim**: 窃取者与被探测目标。**publication**: 把先前写入通过同步协议交给其他线程。**gate**: 共享栈消费者短临界区的占有位。**slot**: 本书对父完成计数中一份活动责任的称呼，不是物理环槽。

sentinel: 提交或用户函数仍在进行时额外持有的一份计数。**anchor**: 完成/异常汇聚的 `Work`。**cache**: 当前调用路径直接接力的一个 `Work` 指针。**quiescent boundary**: 本轮活动责任全部结束、可安全修复下一轮状态的边界。

阅读时先按这条路线确认对象和阶段，再追踪单个原子值的修改者。任何优化都应同时回答：它改变了哪条可见性关系、哪份执行权，以及哪个对象的最后一次访问。

附录：源码指纹与再生成说明

本书基于有未提交修改的工作区。下表给出 SHA-256 前 16 位用于快速识别；完整哈希见同目录 TaskflowLite-Architecture.sources.json。与后续版本比较时，先核对函数名与指纹，再使用本文行号。

核心文件	字节数	SHA-256 前 16 位
async_future.hpp	16520	c5ee21227c9fed1b
async_task.hpp	35287	81f131450411c17d
bounded_queue.hpp	14170	7950fef31a1a6991
executor.hpp	88320	a0ff468aadeee8bc
macros.hpp	24595	475c0483e44d107b
notifier.hpp	55222	78f832e91b007e81
object_pool.hpp	22873	ccb44e3aa1d7d80e
pipeline.hpp	21718	ab22e89c57dc93dc
random.hpp	7555	d72a8cfd0f03f806
result_slot.hpp	11169	5837340f581d28ab
runtime.hpp	31738	08cde2aa0906f7c1
shared_work_stack.hpp	10396	b15325e951b21f23
small_vector.hpp	63934	820df9fd0555b3d3
spin_mutex.hpp	2880	6cae214d4f531dcf
subflow.hpp	3355	4db1b9798ef25b0a
task_group.hpp	30824	23de827ebffa9143
topology.hpp	5924	133612369103b611
work.hpp	76490	920cb8c1f6760df6
work_invokers.hpp	75389	b717b6b069352453
work_storage.hpp	3892	094da603a4e7993a
worker.hpp	7597	1e540a80cdf614be

可编辑源稿为同名 Markdown，所有图解保存在 img/architecture 中的 SVG。

documentation/tools/build_architecture_pdf.py 根据源稿生成 PDF、SVG 和源码指纹；依赖 ReportLab、pypdf 及可嵌入的中文字体。图稿为矢量绘制，可在 PDF 中放大查看。